```
"use client";

import React, { useRef, useState } from 'react';
import {
 Plus, FileText, TrendingUp, ArrowUpRight,
 Users, X, Save, ArrowLeft, FileEdit, Trash2, Clock,
 ChevronDown, Mail, Calendar, Briefcase, Download,
 CheckCircle2, XCircle, Search
} from 'lucide-react';
import Link from 'next/link';

// --- OPTIONS FROM POST JOB SYSTEM ---
const categoryOptions = ["IT & Software", "Design", "Marketing",
"Sales", "Finance", "Human Resources", "Engineering", "Customer
Support", "Operations", "Legal"];
const locationOptions = ["Phnom Penh", "Siem Reap",
"Battambang", "Sihanoukville", "Kampot", "Kandal", "Takeo",
"Remote", "Other"];

// --- COMBOBOX COMPONENT ---
function Combobox({ placeholder, options, inputClass, value,
onChange }: any) {
 const [open, setOpen] = useState(false);
 const safeValue = value || "";
 const filtered = options.filter((o: string) =>
o.toLowerCase().includes(safeValue.toLowerCase()));
  return (
   <div className="relative">
     <input
       type="text"
       value={safeValue}
       placeholder={placeholder}
```

```
        className={`${inputClass} pr-10`}
        onChange={(e) => { onChange(e.target.value);
setOpen(true); }}
        onFocus={() => setOpen(true)}
        onBlur={() => setTimeout(() => setOpen(false), 120)}
      />
      <ChevronDown size={16} className={`absolute right-4 top-1/2
-translate-y-1/2 text-[#6b7f79] pointer-events-none
transition-transform duration-200 ${open ? "rotate-180" : ""}`}
/>
      {open && filtered.length > 0 && (
        <ul className="absolute z-50 mt-1.5 w-full bg-white
border border-gray-200 rounded-xl shadow-lg overflow-hidden
max-h-48 overflow-y-auto">
          {filtered.map((option: string) => (
            <li key={option} onMouseDown={() => {
onChange(option); setOpen(false); }} className="px-4 py-2.5
text-sm font-medium text-[#071a15] cursor-pointer
hover:bg-[#f0f9f6] hover:text-[#40b594]">
              {option}
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}

const EmployerDashboard = () => {
  const [activeView, setActiveView] = useState<"dashboard" |
"drafts">("dashboard");
  const [panelOpen, setPanelOpen] = useState(false);
```

```
 const [panelMode, setPanelMode] = useState<"view" | "edit" |
"view_applicant">("view");
  const [selectedJob, setSelectedJob] = useState<any>({
   title: "", cat: "", loc: "", desc: "", arrangement:
"On-site", employmentType: "Full-time", requirements: ""
 });

 const [selectedApplicant, setSelectedApplicant] =
useState<any>(null);
 const applicantsRef = useRef<HTMLDivElement | null>(null);

 const handleScrollToApplicants = () => {
   applicantsRef.current?.scrollIntoView({ behavior: 'smooth'
});
 };

 // --- DATA ---
 const jobs = [
   { id: 1, title: 'Frontend Developer', cat: 'IT & Software',
loc: 'Phnom Penh', status: 'Active', count: 12, desc: "Building
responsive web apps using React and Next.js." },
   { id: 2, title: 'UI/UX Designer', cat: 'Design', loc:
'Remote', status: 'Active', count: 8, desc: "Designing
user-centric interfaces and prototypes." },
   { id: 3, title: 'Marketing Specialist', cat: 'Marketing',
loc: 'Phnom Penh', status: 'Closed', count: 7, desc: "Running
social media campaigns." },
   { id: 4, title: 'Sales Executive', cat: 'Sales', loc: 'Siem
Reap', status: 'Active', count: 5, desc: "Closing deals and lead
generation." },
 ];
```

```
const applicants = [
  {
      name: 'Rithy Kim', title: 'Frontend Developer', date:
'Apr 23, 2024', status: 'New',
      email: 'rithy.kim@email.com', bio: 'I have 3 years of
experience in React and UI/UX design. I am highly motivated to
build responsive and clean user interfaces.',
      experience: 'Junior Developer at ABC Tech (2021 -
Present)'
  },
  {
      name: 'Nika Yem', title: 'Sales', date: 'Jan 02, 2024',
status: 'New',
      email: 'nika.yem@email.com', bio: 'Specialized in B2B
sales and international market growth with a proven track record
of hitting targets.',
      experience: 'Sales Associate at XYZ Corp'
  },
  {
      name: 'Sokha Chan', title: 'UI/UX Designer', date: 'May
10, 2024', status: 'In Review',
      email: 'sokha.chan@email.com', bio: 'Expert in Figma and
user-centric research. Passionate about creating accessible
digital products.',
      experience: 'Freelance Designer'
  },
];

const statusStyle: Record<string, string> = {
  Active: 'bg-emerald-50 text-emerald-700 ring-1
ring-emerald-200',
  Closed: 'bg-red-50 text-red-700 ring-1 ring-red-200',
```

```
 };

 const applicantStatusStyle: Record<string, string> = {
   New: 'bg-blue-50 text-blue-700 ring-1 ring-blue-200',
   'In Review': 'bg-amber-50 text-amber-700 ring-1
ring-amber-200',
 };

 const openPanel = (job: any, mode: "view" | "edit") => {
   setSelectedJob({ ...job, title: job.title || "", cat: job.cat
|| "", loc: job.loc || "", desc: job.desc || "" });
   setPanelMode(mode);
   setPanelOpen(true);
 };

 const openApplicantDetails = (applicant: any) => {
   setSelectedApplicant(applicant);
   setPanelMode("view_applicant");
   setPanelOpen(true);
 };

 const inputClass = "w-full px-4 py-3 border border-gray-200
rounded-xl bg-[#f8faf9] text-[#071a15] text-sm font-medium
focus:outline-none focus:ring-2 focus:ring-[#40b594]/30
transition-all";
 const labelClass = "block text-sm font-extrabold text-[#071a15]
mb-2";
 const selectClass = "w-full px-4 py-3 border border-gray-200
rounded-xl bg-[#f8faf9] text-[#071a15] text-sm font-medium
focus:outline-none focus:ring-2 focus:ring-[#40b594]/30
transition-all appearance-none";
```

```
 if (activeView === "drafts") {
   return (
     <div className="min-h-screen font-sans pb-16" style={{
background: '#f0f4f3' }}>
       <header className="bg-[#051612] text-white px-8 py-4 flex
items-center justify-between sticky top-0 z-50">
         <div className="flex items-center gap-2.5">
           <img src="/logo.png" alt="NexHire" className="w-8
h-8" />
           <span className="text-xl font-extrabold
tracking-tight">NexHire</span>
         </div>
       </header>
       <main className="max-w-5xl mx-auto px-6 py-12">
         <button onClick={() => setActiveView("dashboard")}
className="flex items-center gap-2 text-[#4a5a55] font-bold mb-8
hover:text-[#071a15] transition-all">
           <ArrowLeft size={20} /> Back to Dashboard
         </button>
         <h1 className="text-4xl font-extrabold text-[#071a15]
mb-8">Drafts & Archive</h1>
         <div className="bg-white p-6 rounded-2xl flex
justify-between items-center shadow-sm border border-gray-100">
             <div className="flex items-center gap-4">
               <div className="w-12 h-12 bg-[#f0f9f6] rounded-xl
flex items-center justify-center border border-[#d1e8e3]">
                 <Clock size={20} className="text-[#40b594]" />
               </div>
               <div>
                 <h3 className="font-extrabold
text-[#071a15]">Data Analyst</h3>
```

```
                <p className="text-xs text-[#6b7f79] font-bold
uppercase">Last saved: 2 hours ago</p>
              </div>
            </div>
            <button className="bg-[#051612] text-white px-6
py-2.5 rounded-xl font-bold text-sm">Resume Posting</button>
        </div>
      </main>
    </div>
  );
}

return (
  <div className="min-h-screen font-sans pb-16 relative
overflow-hidden" style={{ background: '#f0f4f3' }}>

    {/* ─── SLIDE-OVER PANEL ─── */}
    <div className={`fixed inset-0 z-[100] transition-all
duration-300 ${panelOpen ? "opacity-100 visible" : "opacity-0
invisible"}`}>
      <div className="absolute inset-0 bg-black/20
backdrop-blur-sm" onClick={() => setPanelOpen(false)} />
      <div className={`absolute right-0 top-0 h-full w-full
max-w-2xl bg-white shadow-2xl p-0 overflow-hidden transform
transition-transform duration-300 ${panelOpen ? "translate-x-0"
: "translate-x-full"}`}>

        <div className="h-full flex flex-col">
          <div className="px-10 py-8 border-b border-gray-100
flex justify-between items-center bg-white sticky top-0 z-10">
              <div className="flex items-center gap-5">
                {panelMode === "view_applicant" ? (
```

```
                        <div className="w-16 h-16 rounded-2xl
bg-[#051612] flex items-center justify-center text-white
text-2xl font-black">
                            {selectedApplicant?.name.charAt(0)}
                        </div>
                    ) : (
                        <div className="w-16 h-16 rounded-2xl
bg-[#f0f9f6] flex items-center justify-center text-[#40b594]
border border-[#d1e8e3]">
                            <FileEdit size={28} />
                        </div>
                    )}
                    <div>
                        <p className="text-[10px] font-black
uppercase tracking-widest text-[#40b594] mb-1">
                            {panelMode === "view_applicant" ?
"Job Application" : "Edit Listing"}
                        </p>
                        <h2 className="text-2xl font-extrabold
text-[#071a15]">
                            {panelMode === "view_applicant" ?
selectedApplicant?.name : selectedJob.title}
                        </h2>
                    </div>
                </div>
                <button onClick={() => setPanelOpen(false)}
className="p-2 hover:bg-gray-100 rounded-full text-gray-400"><X
size={24} /></button>
            </div>

            <div className="flex-1 overflow-y-auto p-10 pt-6">
                {panelMode === "view_applicant" ? (
```

```
                    <div className="space-y-8">
                        <div className="grid grid-cols-2 gap-4">
                            <div className="bg-[#f8faf9] p-5
rounded-2xl border border-gray-100">
                                <p className="text-[10px]
font-black text-[#6b7f79] uppercase mb-1 flex items-center
gap-1"><Mail size={12}/> Email Address</p>
                                <p className="text-sm font-bold
text-[#071a15]">{selectedApplicant?.email}</p>
                            </div>
                            <div className="bg-[#f8faf9] p-5
rounded-2xl border border-gray-100">
                                <p className="text-[10px]
font-black text-[#6b7f79] uppercase mb-1 flex items-center
gap-1"><Calendar size={12}/> Applied Date</p>
                                <p className="text-sm font-bold
text-[#071a15]">{selectedApplicant?.date}</p>
                            </div>
                        </div>

                        <div>
                            <h4 className="text-xs font-black
text-[#071a15] uppercase tracking-wider mb-3 flex items-center
gap-2">
                                <Briefcase size={16}
className="text-[#40b594]"/> Last Experience
                            </h4>
                            <div className="p-5 bg-white border
border-gray-100 rounded-2xl shadow-sm">
                                <p className="text-sm font-bold
text-[#071a15]">{selectedApplicant?.title}</p>
```

```
                                <p className="text-xs
font-semibold text-[#6b7f79]
mt-1">{selectedApplicant?.experience}</p>
                            </div>
                        </div>

                        <div>
                            <h4 className="text-xs font-black
text-[#071a15] uppercase tracking-wider mb-3 flex items-center
gap-2">
                                <FileText size={16}
className="text-[#40b594]"/> Cover Letter / Bio
                            </h4>
                            <p className="text-sm text-[#4a5a55]
leading-relaxed font-medium bg-[#f0f9f6]/40 p-6 rounded-2xl
border border-[#d1e8e3]">
                                {selectedApplicant?.bio}
                            </p>
                        </div>

                        <div className="p-5 bg-white border-2
border-dashed border-[#d1e8e3] rounded-2xl flex items-center
justify-between">
                            <div className="flex items-center
gap-3">
                                <div className="w-10 h-10
bg-red-50 rounded-lg flex items-center justify-center
text-red-500 font-black text-[10px]">PDF</div>
                                <div>
                                    <p className="text-sm
font-bold text-[#071a15]">Resume_CV.pdf</p>
```

```
                                    <p className="text-[10px]
font-bold text-[#6b7f79]">1.2 MB</p>
                                </div>
                            </div>
                            <button className="flex items-center
gap-2 text-[#40b594] font-black text-xs hover:bg-[#f0f9f6] p-2
rounded-lg transition-all">
                                <Download size={18} />
                            </button>
                        </div>
                    </div>
                ) : (
                    <div className="space-y-6">
                        <div>
                            <label className={labelClass}>Job
Title <span className="text-red-500">*</span></label>
                            <input value={selectedJob.title}
className={inputClass} onChange={(e) =>
setSelectedJob({...selectedJob, title: e.target.value})} />
                        </div>
                        <div className="grid grid-cols-2 gap-6">
                            <div><label
className={labelClass}>Category</label><Combobox
options={categoryOptions} inputClass={inputClass}
value={selectedJob.cat}
onChange={(v:any)=>setSelectedJob({...selectedJob, cat:v})}
/></div>
                            <div><label
className={labelClass}>Location</label><Combobox
options={locationOptions} inputClass={inputClass}
value={selectedJob.loc}
```

```
onChange={(v:any)=>setSelectedJob({...selectedJob, loc:v})}
/></div>
                        </div>
                        <div className="grid grid-cols-2 gap-6">
                            <div className="relative">
                                <label
className={labelClass}>Work Arrangement</label>
                                <select className={selectClass}
value={selectedJob.arrangement} onChange={(e) =>
setSelectedJob({...selectedJob, arrangement: e.target.value})}>

<option>On-site</option><option>Remote</option><option>Hybrid</o
ption>
                                </select>
                                <ChevronDown size={16}
className="absolute right-4 top-[42px] text-[#6b7f79]
pointer-events-none" />
                            </div>
                            <div className="relative">
                                <label
className={labelClass}>Employment Type</label>
                                <select className={selectClass}
value={selectedJob.employmentType} onChange={(e) =>
setSelectedJob({...selectedJob, employmentType:
e.target.value})}>

<option>Full-time</option><option>Part-time</option><option>Cont
ract</option><option>Freelance</option>
                                </select>
                                <ChevronDown size={16}
className="absolute right-4 top-[42px] text-[#6b7f79]
pointer-events-none" />
```

```
                            </div>
                        </div>
                        <div>
                            <label
className={labelClass}>Description</label>
                            <textarea rows={6}
value={selectedJob.desc} className={`${inputClass} resize-none`}
onChange={(e) => setSelectedJob({...selectedJob, desc:
e.target.value})} />
                        </div>
                        <button className="w-full bg-[#051612]
text-white py-4 rounded-xl font-extrabold text-sm flex
items-center justify-center gap-2 hover:bg-[#0d2a23]
transition-all"><Save size={18} /> Save Changes</button>
                    </div>
                )}
            </div>

            {panelMode === "view_applicant" && (
                <div className="p-8 border-t border-gray-100
bg-[#f8faf9] flex items-center gap-3">
                    <button className="flex-1 bg-white border
border-gray-200 text-[#4a5a55] py-4 rounded-2xl font-black
text-[10px] tracking-widest flex items-center justify-center
gap-2 hover:bg-amber-50 hover:text-amber-600
hover:border-amber-200 transition-all uppercase">
                        <Search size={14} /> Under Review
                    </button>
                    <button className="flex-1 bg-white border
border-gray-200 text-[#4a5a55] py-4 rounded-2xl font-black
text-[10px] tracking-widest flex items-center justify-center
```

```
gap-2 hover:bg-red-50 hover:text-red-600 hover:border-red-200
transition-all uppercase">
                        <XCircle size={14} /> Reject
                    </button>
                    <button className="flex-[1.5] bg-[#051612]
text-white py-4 rounded-2xl font-black text-[10px]
tracking-widest flex items-center justify-center gap-2
hover:bg-[#40b594] transition-all shadow-lg uppercase">
                        <CheckCircle2 size={14} /> Accept
Applicant
                    </button>
                </div>
            )}
          </div>
        </div>
      </div>

      <header className="bg-[#051612] text-white px-8 py-4 flex
items-center justify-between sticky top-0 z-50">
        <div className="flex items-center gap-2.5">
          <img src="/logo.png" alt="NexHire" className="w-8 h-8"
/>
          <span className="text-xl font-extrabold
tracking-tight">NexHire</span>
        </div>
        <nav className="hidden md:flex items-center gap-8 text-sm
font-semibold">
          <button className="text-[#40b594] border-b-2
border-[#40b594] pb-1">Dashboard</button>
          <Link href="/post_job"><button className="text-gray-300
hover:text-white transition-colors">Post Job</button></Link>
```

```
        <Link href="/employer_message"><button
className="text-gray-300 hover:text-white
transition-colors">Messages</button></Link>
        <Link href="/employer_notification"><button
className="text-gray-300 hover:text-white
transition-colors">Notification</button></Link>
        <Link href="/subscription"><button
className="text-gray-300 hover:text-white
transition-colors">Subscription</button></Link>
        <Link href="/employer_setting"><button
className="text-gray-300 hover:text-white
transition-colors">Settings</button></Link>
      </nav>
      <Link href="/employer_profile">
        <div className="flex items-center gap-3 cursor-pointer
group">
          <div className="text-right">
            <p className="text-[10px] text-gray-500 uppercase
tracking-widest">Company</p>
            <p className="text-sm font-bold text-white
group-hover:text-[#40b594] transition-colors">Profile</p>
          </div>
          <div className="w-10 h-10 bg-[#40b594] rounded-full
flex items-center justify-center font-extrabold text-[#051612]
text-sm">C</div>
        </div>
      </Link>
    </header>

    <main className="max-w-7xl mx-auto px-6 md:px-10 py-10">
      <div className="flex items-center justify-between mb-10">
        <div>
```

```
          <p className="text-xs font-bold uppercase
tracking-widest text-[#40b594] mb-1">Dashboard Overview</p>
          <h1 className="text-4xl font-extrabold text-[#071a15]
leading-tight">Welcome back, Company1</h1>
        </div>
        <Link href="/post_job">
          <button className="flex items-center gap-2
bg-[#051612] text-white px-6 py-3 rounded-xl font-bold text-sm
hover:bg-[#0d2a23] transition-all">
            <Plus size={18} /> Post New Job
          </button>
        </Link>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-3 gap-5
mb-8">
        {[
          { label: 'Total Jobs Posted', value: '5', icon:
FileText, bg: 'bg-[#051612]', iconBg: 'bg-[#0d2a23]', trend: '+2
this month' },
          { label: 'Total Applicants', value: '32', icon:
Users, bg: 'bg-[#133228]', iconBg: 'bg-[#1a4035]', trend: '+8
this week' },
          { label: 'Active Jobs', value: '3', icon: TrendingUp,
bg: 'bg-[#40b594]', iconBg: 'bg-[#33997a]', trend: '3 open
roles', dark: true },
        ].map(({ label, value, icon: Icon, bg, iconBg, trend,
dark }) => (
          <div key={label} className={`${bg} rounded-2xl p-7
flex flex-col gap-5`}>
            <div className="flex items-start justify-between">
```

```
                <div className={`${iconBg} p-3 rounded-xl`}><Icon
className={dark ? 'text-[#051612]' : 'text-[#40b594]'} size={24}
/></div>
                <ArrowUpRight className={dark ? 'text-[#051612]
opacity-60' : 'text-[#40b594] opacity-60'} size={18} />
              </div>
              <div>
                <p className={`text-5xl font-extrabold ${dark ?
'text-[#051612]' : 'text-white'} leading-none mb-1`}>{value}</p>
                <p className={`text-sm font-semibold ${dark ?
'text-[#071a15]' : 'text-gray-300'}`}>{label}</p>
              </div>
            </div>
          ))}
        </div>

        <div className="grid grid-cols-1 lg:grid-cols-3 gap-6
mb-6">
          <div className="lg:col-span-2 bg-white rounded-2xl
shadow-sm border border-gray-100 overflow-hidden">
            <div className="flex items-center justify-between
px-8 py-6 border-b border-gray-100"><h2 className="text-lg
font-extrabold text-[#071a15]">Recent Jobs</h2></div>
            <div className="overflow-x-auto">
              <table className="w-full text-left">
                <thead>
                  <tr className="bg-[#f8faf9] text-xs font-bold
uppercase tracking-wider text-[#6b7f79]">
                    <th className="px-8 py-4">Job Title</th><th
className="px-4 py-4">Category</th><th className="px-4
py-4">Location</th><th className="px-4 py-4">Status</th><th
```

```
className="px-4 py-4 text-center">Applicants</th><th
className="px-4 py-4 text-right pr-8">Actions</th>
                </tr>
              </thead>
              <tbody>
                {jobs.map((job, idx) => (
                  <tr key={idx} className="border-t
border-gray-50 hover:bg-[#f8faf9] transition-colors group">
                    <td className="px-8 py-4 font-bold
text-[#071a15] text-sm">{job.title}</td>
                    <td className="px-4 py-4 text-[#4a5a55]
text-sm font-medium">{job.cat}</td>
                    <td className="px-4 py-4 text-[#4a5a55]
text-sm font-medium">{job.loc}</td>
                    <td className="px-4 py-4"><span
className={`px-3 py-1 rounded-full text-xs font-bold
${statusStyle[job.status]}`}>{job.status}</span></td>
                    <td className="px-4 py-4 text-center
font-extrabold text-[#071a15] text-sm">{job.count}</td>
                    <td className="px-4 py-4 text-right
pr-8"><div className="flex justify-end gap-1"><button
onClick={() => openPanel(job, "edit")} className="p-2
text-gray-400 hover:text-[#40b594] transition-all"><FileEdit
size={16}/></button><button className="p-2 text-gray-400
hover:text-red-500 transition-all"><Trash2
size={16}/></button></div></td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        </div>
```

```
        <div className="bg-white rounded-2xl shadow-sm border
border-gray-100 p-6 flex flex-col gap-4 h-fit sticky top-24">
          <h2 className="text-lg font-extrabold text-[#071a15]
mb-2">Quick Actions</h2>
          <Link href="/post_job" className="block"><button
className="w-full bg-[#051612] text-white py-4 rounded-xl
font-bold text-sm flex items-center justify-center gap-2.5
hover:bg-[#0d2a23] transition-all"><Plus size={18} /> Post a New
Job</button></Link>
          <button onClick={handleScrollToApplicants}
className="w-full border-2 border-[#d1e8e3] text-[#071a15] py-4
rounded-xl font-bold text-sm flex items-center justify-center
gap-2.5 hover:bg-[#f0f9f6] hover:border-[#40b594]
transition-all"><Users size={18} /> View Applicants</button>
          <button onClick={() => setActiveView("drafts")}
className="w-full border-2 border-[#d1e8e3] text-[#071a15] py-4
rounded-xl font-bold text-sm flex items-center justify-center
gap-2.5 hover:bg-[#f0f9f6] hover:border-[#40b594]
transition-all"><FileText size={18} /> Drafts</button>
        </div>
      </div>

      <section ref={applicantsRef} className="bg-white
rounded-2xl shadow-sm border border-gray-100 overflow-hidden">
        <div className="flex items-center justify-between px-8
py-6 border-b border-gray-100">
          <h2 className="text-lg font-extrabold
text-[#071a15]">Recent Applicants</h2>
          <span className="text-xs font-bold text-[#40b594]
bg-emerald-50 px-3 py-1 rounded-full">{applicants.length}
new</span>
        </div>
```

```
        <table className="w-full text-left">
          <thead>
            <tr className="bg-[#f8faf9] text-xs font-bold
uppercase tracking-wider text-[#6b7f79]">
              <th className="px-8 py-4">Name</th>
              <th className="px-4 py-4">Applied For</th>
              <th className="px-4 py-4">Date</th>
              <th className="px-4 py-4">Status</th>
              <th className="px-4 py-4"></th>
            </tr>
          </thead>
          <tbody>
            {applicants.map((a, idx) => (
              <tr key={idx} className="border-t border-gray-50
hover:bg-[#f8faf9] transition-colors">
                <td className="px-8 py-5">
                  <div className="flex items-center gap-3">
                    <div className="w-8 h-8 rounded-full
bg-[#d1e8e3] flex items-center justify-center text-xs
font-extrabold text-[#051612]">{a.name.charAt(0)}</div>
                    <span className="font-bold text-[#071a15]
text-sm">{a.name}</span>
                  </div>
                </td>
                <td className="px-4 py-5 text-[#4a5a55] text-sm
font-medium">{a.title}</td>
                <td className="px-4 py-5 text-[#6b7f79]
text-sm">{a.date}</td>
                <td className="px-4 py-5"><span
className={`px-3 py-1 rounded-full text-[10px] font-black
uppercase ${applicantStatusStyle[a.status] ??
'bg-gray-100'}`}>{a.status}</span></td>
```

```
                  <td className="px-4 py-5 text-right pr-8">
                    <button onClick={() =>
openApplicantDetails(a)} className="text-xs font-black
text-[#40b594] hover:bg-[#f0f9f6] px-3 py-1.5 rounded-lg
transition-all">View →</button>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </section>
      </main>
    </div>
  );
};

export default EmployerDashboard;
"use client";

import React, { useState } from 'react';
import { Search, Paperclip, Send } from 'lucide-react';
import Link from 'next/link';

const Messages = () => {
  const [activeChat, setActiveChat] = useState(1);
  const [message, setMessage] = useState('');

  const chatList = [
    {
      id: 1,
      name: 'Marady',
      role: 'Engineering Manager',
```

```
    time: '10:30 PM',
    initial: 'M',
    unread: 2,
    status: 'Online',
  },
  {
    id: 2,
    name: 'Mars',
    role: 'Recruiter at Stripe',
    time: 'Yesterday',
    initial: 'M',
    unread: 0,
    status: 'Away',
  },
];

const activeContact = chatList.find((c) => c.id ===
activeChat);

return (
  <div className="min-h-screen bg-[#f0f4f3] font-sans">
    <header className="bg-[#051612] text-white px-8 py-4 flex
items-center justify-between sticky top-0 z-50 shadow-lg">
      <div className="flex items-center gap-2.5">
        <img src="/logo.png" alt="NexHire" className="w-8 h-8"
/>
        <span className="text-xl font-extrabold
tracking-tight">NexHire</span>
      </div>

      <nav className="hidden md:flex items-center gap-8 text-sm
font-semibold">
```

```
        <Link href="/dashboard">
          <button className="text-gray-300 hover:text-white
transition-colors">Dashboard</button>
        </Link>
        <Link href="/post_job">
          <button className="text-gray-300 hover:text-white
transition-colors">Post Job</button>
        </Link>
        <button className="text-[#40b594] border-b-2
border-[#40b594] pb-1">Messages</button>
        <Link href="/employer_notification">
          <button className="text-gray-300 hover:text-white
transition-colors">Notification</button>
        </Link>
        <Link href="/subscription">
          <button className="text-gray-300 hover:text-white
transition-colors">Subscription</button>
        </Link>
        <Link href="/employer_setting">
          <button className="text-gray-300 hover:text-white
transition-colors">Settings</button>
        </Link>
      </nav>

      <Link href="/emprofile">
        <div className="flex items-center gap-3 cursor-pointer
group">
          <div className="text-right">
            <p className="text-[10px] text-gray-500 uppercase
tracking-widest">Company</p>
            <p className="text-sm font-bold text-white
group-hover:text-[#40b594] transition-colors">Profile</p>
```

```
          </div>
          <div className="w-10 h-10 bg-[#40b594] rounded-full
flex items-center justify-center font-extrabold text-[#051612]
text-sm">C</div>
        </div>
      </Link>
    </header>

    <main className="mx-auto max-w-7xl px-6 py-10 md:px-10">
      <div className="mb-8">
        <p className="mb-1 text-xs font-bold uppercase
tracking-widest text-[#40b594]">Inbox</p>
        <h1 className="text-4xl font-extrabold
text-[#071a15]">Messages</h1>
        <p className="mt-1 font-medium text-[#4a5a55]">
          Communicate with recruiters and hiring managers
        </p>
      </div>

      <div className="flex h-[680px] overflow-hidden
rounded-2xl border border-gray-100 bg-white shadow-sm">
        <div className="flex w-80 flex-shrink-0 flex-col
border-r border-gray-100">
          <div className="border-b border-gray-100 p-5">
            <div className="relative">
              <Search
                className="absolute left-3.5 top-1/2
-translate-y-1/2 text-[#6b7f79]"
                size={16}
              />
              <input
                type="text"
```

```
                placeholder="Search conversations..."
                className="w-full rounded-xl bg-[#f0f4f3]
py-2.5 pl-10 pr-4 text-sm font-medium text-[#071a15]
placeholder-[#6b7f79] focus:outline-none focus:ring-2
focus:ring-[#40b594]/30"
              />
            </div>
          </div>

          <div className="px-5 py-3">
            <p className="text-[10px] font-bold uppercase
tracking-widest text-[#6b7f79]">
              Recent
            </p>
          </div>

          <div className="flex-1 overflow-y-auto">
            {chatList.map((chat) => {
              const isActive = activeChat === chat.id;

              return (
                <div
                  key={chat.id}
                  onClick={() => setActiveChat(chat.id)}
                  className={`cursor-pointer border-l-4 px-5
py-4 transition-all ${
                    isActive
                      ? 'border-l-[#40b594] bg-[#f0f9f6]'
                      : 'border-l-transparent
hover:bg-[#f8faf9]'
                  }`}
                >
```

```
                    <div className="flex items-center gap-4">
                      <div className="relative flex-shrink-0">
                        <div className="flex h-12 w-12
items-center justify-center rounded-full bg-[#051612] text-base
font-extrabold text-white">
                          {chat.initial}
                        </div>
                        <span
                          className={`absolute bottom-0 right-0
h-3 w-3 rounded-full border-2 border-white ${
                            chat.status === 'Online' ?
'bg-[#40b594]' : 'bg-[#d7b14a]'
                          }`}
                        />
                      </div>

                      <div className="min-w-0 flex-1">
                        <div className="mb-0.5 flex items-center
justify-between">
                          <h3
                            className={`truncate text-sm
font-extrabold ${
                              isActive ? 'text-[#071a15]' :
'text-[#1a2e29]'
                            }`}
                          >
                            {chat.name}
                          </h3>
                          <span className="ml-2 flex-shrink-0
text-[10px] font-semibold text-[#6b7f79]">
                            {chat.time}
                          </span>
```

```
                      </div>

                      <p className="truncate text-xs font-bold
tracking-wide text-[#40b594]">
                        {chat.role}
                      </p>

                      <p className="mt-1 text-[11px]
font-semibold text-[#7a8b86]">
                        {chat.status}
                      </p>
                    </div>

                    {chat.unread > 0 && (
                      <span className="flex h-5 w-5
flex-shrink-0 items-center justify-center rounded-full
bg-[#40b594] text-[10px] font-extrabold text-white">
                        {chat.unread}
                      </span>
                    )}
                  </div>
                </div>
              );
            })}
          </div>
        </div>

        <div className="flex min-w-0 flex-1 flex-col">
          <div className="border-b border-gray-100 bg-white
px-7 py-5">
            <div className="flex items-center gap-4">
              <div className="relative flex-shrink-0">
```

```
                  <div className="flex h-12 w-12 items-center
justify-center rounded-full bg-[#051612] text-sm font-extrabold
text-white">
                    {activeContact?.initial}
                  </div>
                  <span
                    className={`absolute bottom-0 right-0 h-3 w-3
rounded-full border-2 border-white ${
                      activeContact?.status === 'Online' ?
'bg-[#40b594]' : 'bg-[#d7b14a]'
                    }`}
                  />
                </div>

                <div className="min-w-0">
                  <h3 className="truncate text-base
font-extrabold text-[#071a15]">
                    {activeContact?.name}
                  </h3>
                  <p className="truncate text-sm font-medium
text-[#4f655f]">
                    {activeContact?.role}
                  </p>
                  <p className="mt-1 text-[11px] font-bold
uppercase tracking-wider text-[#40b594]">
                    {activeContact?.status}
                  </p>
                </div>
              </div>
            </div>
```

```
          <div className="flex-1 space-y-6 overflow-y-auto
bg-[#f8faf9] px-7 py-6">
            <div className="flex items-center gap-4">
              <div className="h-px flex-1 bg-gray-200" />
              <span className="text-[10px] font-bold uppercase
tracking-widest text-[#6b7f79]">
                Today
              </span>
              <div className="h-px flex-1 bg-gray-200" />
            </div>

            <div className="flex items-end gap-3">
              <div className="flex h-8 w-8 flex-shrink-0
items-center justify-center rounded-full bg-[#051612] text-xs
font-extrabold text-white">
                {activeContact?.initial}
              </div>
              <div className="flex flex-col items-start gap-1">
                <div className="max-w-sm rounded-2xl
rounded-bl-md bg-[#051612] px-5 py-3.5 text-white shadow-sm">
                  <p className="text-sm leading-relaxed">
                    Hi! Are you still available for the
interview on Monday at 4:30 PM?
                  </p>
                </div>
                <span className="ml-1 text-[10px] font-semibold
text-[#6b7f79]">10:20 AM</span>
              </div>
            </div>

            <div className="flex flex-col items-end gap-1">
```

```
                <div className="max-w-sm rounded-2xl
rounded-br-md border border-gray-200 bg-white px-5 py-3.5
shadow-sm">
                  <p className="text-sm leading-relaxed
text-[#071a15]">
                    Yes, absolutely! I have it on my calendar.
Looking forward to it.
                  </p>
                </div>
                <div className="mr-1 flex items-center gap-1.5">
                  <span className="text-[10px] font-semibold
text-[#6b7f79]">10:28 AM</span>
                  <svg
                    width="14"
                    height="14"
                    viewBox="0 0 24 24"
                    fill="none"
                    stroke="#40b594"
                    strokeWidth="3"
                    strokeLinecap="round"
                    strokeLinejoin="round"
                  >
                    <polyline points="20 6 9 17 4 12" />
                  </svg>
                </div>
              </div>
            </div>

            <div className="border-t border-gray-100 bg-white
px-7 py-5">
              <div className="flex items-center gap-3 rounded-2xl
border border-gray-200 bg-[#f0f4f3] px-4 py-3 transition-all
```

```
focus-within:border-[#40b594] focus-within:ring-2
focus-within:ring-[#40b594]/20">
               <button className="flex-shrink-0 text-[#6b7f79]
transition-colors hover:text-[#071a15]">
                 <Paperclip size={19} />
               </button>
               <input
                 type="text"
                 placeholder="Write a message..."
                 value={message}
                 onChange={(e) => setMessage(e.target.value)}
                 className="flex-1 border-none bg-transparent
text-sm font-medium text-[#071a15] placeholder-[#6b7f79]
outline-none"
               />
               <button
                 className={`flex-shrink-0 rounded-xl p-2
transition-all ${
                   message.trim()
                     ? 'bg-[#051612] text-white shadow-sm
hover:bg-[#0d2a23]'
                     : 'cursor-not-allowed bg-[#d1e8e3]
text-[#6b7f79]'
                 }`}
               >
                 <Send size={17} />
               </button>
             </div>
           </div>
         </div>
       </div>
     </main>
```

```
   </div>
 );
};

export default Messages;
"use client";

import React, { useState } from 'react';
import { CheckCircle, Clock, MoreVertical, ArrowRight, Bell,
User, Zap } from 'lucide-react';
import Link from 'next/link';

const Notifications = () => {
 const [notifications, setNotifications] = useState([
   {
     id: 1,
     type: 'Applicant',
     icon: User,
     title: 'Sok Rathana applied for Frontend Developer',
     description: 'New applicant profile ready for your
review.',
     time: '20m ago',
     actionText: 'View Profile',
     read: false,
   },
   {
     id: 2,
     type: 'NexHire',
     icon: Zap,
     title: 'Your Standard Plan has been activated!',
     description: 'You now have access to 3 active job slots per
month.',
```

```
    time: '1h ago',
    actionText: 'View Billing',
    read: false,
  },
  {
    id: 3,
    type: 'Applicant',
    icon: User,
    title: 'Pi Sari applied for Data Analyst',
    description: 'New applicant profile ready for your
review.',
    time: '2h ago',
    actionText: 'View Profile',
    read: true,
  },
 ]);

 const unreadCount = notifications.filter(n => !n.read).length;

 const markAllRead = () => {
   setNotifications(prev => prev.map(n => ({ ...n, read: true
})));
 };

 const markOneRead = (id: number) => {
   setNotifications(prev => prev.map(n => n.id === id ? { ...n,
read: true } : n));
 };

 const typeStyle: Record<string, { badge: string; iconBg:
string; iconColor: string }> = {
   Applicant: {
```

```
    badge:     'bg-blue-50 text-blue-700 ring-1 ring-blue-200',
    iconBg:    'bg-[#051612]',
    iconColor: 'text-[#40b594]',
  },
  NexHire: {
    badge:     'bg-emerald-50 text-emerald-700 ring-1
ring-emerald-200',
    iconBg:    'bg-[#40b594]',
    iconColor: 'text-[#051612]',
  },
};

return (
  <div className="min-h-screen font-sans pb-16" style={{
background: '#f0f4f3' }}>

    {/* ── HEADER ── */}
    <header className="bg-[#051612] text-white px-8 py-4 flex
items-center justify-between sticky top-0 z-50 shadow-lg">
      <div className="flex items-center gap-2.5">
        <img src="/logo.png" alt="NexHire" className="w-8 h-8"
/>
        <span className="text-xl font-extrabold
tracking-tight">NexHire</span>
      </div>

      <nav className="hidden md:flex items-center gap-8 text-sm
font-semibold">
        <Link href="/dashboard">
          <button className="text-gray-300 hover:text-white
transition-colors">Dashboard</button>
        </Link>
```

```
        <Link href="/post_job">
          <button className="text-gray-300 hover:text-white
transition-colors">Post Job</button>
        </Link>
        <Link href="/employer_message">
          <button className="text-gray-300 hover:text-white
transition-colors">Messages</button>
        </Link>
        <button className="text-[#40b594] border-b-2
border-[#40b594] pb-1">Notification</button>
        <Link href="/subscription">
          <button className="text-gray-300 hover:text-white
transition-colors">Subscription</button>
        </Link>
        <Link href="/employer_setting">
          <button className="text-gray-300 hover:text-white
transition-colors">Settings</button>
        </Link>
      </nav>

      <Link href="/employer_profile">
        <div className="flex items-center gap-3 cursor-pointer
group">
          <div className="text-right">
            <p className="text-[10px] text-gray-500 uppercase
tracking-widest">Company</p>
            <p className="text-sm font-bold text-white
group-hover:text-[#40b594] transition-colors">Profile</p>
          </div>
          <div className="w-10 h-10 bg-[#40b594] rounded-full
flex items-center justify-center font-extrabold text-[#051612]
text-sm">C</div>
```

```
          </div>
        </Link>
      </header>

      <main className="max-w-3xl mx-auto px-6 md:px-10 py-10">

        {/* ── PAGE TITLE ── */}
        <div className="flex justify-between items-start mb-10">
          <div>
            <p className="text-xs font-bold uppercase
tracking-widest text-[#40b594] mb-1">Activity</p>
            <h1 className="text-4xl font-extrabold text-[#071a15]
leading-tight">Notifications</h1>
            <p className="text-[#4a5a55] font-medium mt-1">Keep
track of your hiring activity</p>
          </div>

          <div className="flex items-center gap-3 mt-2">
            {unreadCount > 0 && (
              <div className="flex items-center gap-2 bg-white
border border-gray-200 rounded-xl px-4 py-2 shadow-sm">
                <Bell size={15} className="text-[#40b594]" />
                <span className="text-sm font-extrabold
text-[#071a15]">{unreadCount} unread</span>
              </div>
            )}
            <button
              onClick={markAllRead}
              className="flex items-center gap-2 bg-white border
border-gray-200 text-[#071a15] px-4 py-2 rounded-xl font-bold
text-sm hover:border-[#40b594] hover:text-[#40b594]
transition-all shadow-sm"
```

```
          >
            <CheckCircle size={16} />
            Mark all read
          </button>
        </div>
      </div>

      {/* ── NOTIFICATION LIST ── */}
      <div className="space-y-4">
        {notifications.map((notif) => {
          const style = typeStyle[notif.type] ??
typeStyle['Applicant'];
          const Icon = notif.icon;

          return (
            <div
              key={notif.id}
              className={`bg-white rounded-2xl border
transition-all shadow-sm overflow-hidden ${
                notif.read
                  ? 'border-gray-100'
                  : 'border-l-4 border-l-[#40b594]
border-t-gray-100 border-r-gray-100 border-b-gray-100'
              } hover:shadow-md hover:border-l-[#40b594]`}
            >
              <div className="p-7">

                <div className="flex items-start
justify-between mb-4">
                  <div className="flex items-center gap-3">
```

```
                    <div className={`w-10 h-10 rounded-xl
${style.iconBg} flex items-center justify-center
flex-shrink-0`}>
                      <Icon size={18}
className={style.iconColor} />
                    </div>

                    <div>
                      <span className={`text-xs font-extrabold
px-2.5 py-1 rounded-full ${style.badge}`}>
                        {notif.type}
                      </span>
                      <div className="flex items-center gap-1.5
mt-1.5">
                        <Clock size={12}
className="text-[#6b7f79]" />
                        <span className="text-xs font-semibold
text-[#6b7f79]">{notif.time}</span>
                      </div>
                    </div>
                  </div>

                  <div className="flex items-center gap-2">
                    {!notif.read && (
                      <span className="w-2.5 h-2.5 rounded-full
bg-[#40b594] flex-shrink-0" />
                    )}
                    <button
                      onClick={() => markOneRead(notif.id)}
                      className="p-1.5 rounded-lg
text-[#6b7f79] hover:bg-[#f0f4f3] hover:text-[#071a15]
transition-all"
```

```
                    >
                      <MoreVertical size={18} />
                    </button>
                  </div>
                </div>

                <div className="pl-13 ml-1">
                  <h3 className="text-base font-extrabold
text-[#071a15] leading-snug mb-1">
                    {notif.title}
                  </h3>
                  <p className="text-sm text-[#4a5a55]
font-medium leading-relaxed">
                    {notif.description}
                  </p>
                </div>

                <div className="mt-5 flex items-center
justify-between">
                  <button className="flex items-center gap-2
bg-[#051612] text-white px-5 py-2.5 rounded-xl font-bold text-sm
hover:bg-[#0d2a23] transition-all shadow-sm">
                    {notif.actionText}
                    <ArrowRight size={16} />
                  </button>

                  {!notif.read && (
                    <button
                      onClick={() => markOneRead(notif.id)}
                      className="text-xs font-bold
text-[#6b7f79] hover:text-[#40b594] transition-colors"
                    >
```

```
                        Mark as read
                      </button>
                    )}
                  </div>
                </div>
              </div>
            );
          })}
        </div>

        {notifications.length === 0 && (
          <div className="text-center py-24">
            <div className="w-16 h-16 bg-white rounded-2xl flex
items-center justify-center mx-auto mb-4 shadow-sm border
border-gray-100">
              <Bell size={28} className="text-[#40b594]" />
            </div>
            <p className="text-lg font-extrabold
text-[#071a15]">All caught up!</p>
            <p className="text-sm text-[#4a5a55] mt-1">No new
notifications right now.</p>
          </div>
        )}

      </main>
    </div>
  );
};

export default Notifications;
// app/employer_profile/page.tsx
"use client";
```

```
import React from "react";
import Link from "next/link";
import {
 Globe,
 MapPin,
 Users,
 Mail,
 Phone,
 Briefcase,
 CheckCircle2,
 Building2,
 Pencil,
 TrendingUp,
 Save,
 X,
 Sparkles,
 Camera,
 Trash2,
 FileText,
 Upload,
} from "lucide-react";
import ImageCropModal from "@/components/ui/ImageCropModal";

type EmployerForm = {
 companyDescription: string;
 companyName: string;
 industry: string;
 companySize: string;
 currentAddress: string;
 foundedYear: string;
 country: string;
```

```
 contact: string;
 websiteLink: string;
 companyFileName: string;
 companyFileUrl: string;
 profileImageUrl: string;
};

const cardClass =
 "rounded-[28px] border border-[#e6eeea] bg-white
shadow-[0_18px_50px_rgba(7,28,22,0.06)]";

export default function EmployerProfile() {
 const fileInputRef = React.useRef<HTMLInputElement |
null>(null);
 const companyFileInputRef = React.useRef<HTMLInputElement |
null>(null);

 const [activeTab, setActiveTab] = React.useState<"overview" |
"details">("overview");
 const [isEditing, setIsEditing] = React.useState(false);
 const [loading, setLoading] = React.useState(false);

 const [profileImage, setProfileImage] = React.useState<File |
null>(null);
 const [sourceImageUrl, setSourceImageUrl] =
React.useState<string | null>(null);
 const [croppedPreviewUrl, setCroppedPreviewUrl] =
React.useState<string | null>(null);
 const [cropOpen, setCropOpen] = React.useState(false);

 const [companyFile, setCompanyFile] = React.useState<File |
null>(null);
```

```
 const [saved, setSaved] = React.useState<EmployerForm>({
   companyDescription:
     "NexHire is a modern hiring company focused on helping
employers find qualified talent faster with a cleaner and more
human recruitment experience.",
   companyName: "NexHire Solutions",
   industry: "Technology & Software",
   companySize: "50 - 100 Employees",
   currentAddress: "Phnom Penh, Cambodia",
   foundedYear: "2021",
   country: "Cambodia",
   contact: "+855 23 456 789",
   websiteLink: "https://www.nexhire.com",
   companyFileName: "",
   companyFileUrl: "",
   profileImageUrl: "",
 });

 const [draft, setDraft] = React.useState<EmployerForm>(saved);

 const stats = [
   { label: "Active Jobs", value: "12", icon: Briefcase },
   { label: "Hires", value: "148", icon: Users },
   { label: "Growth", value: "+26%", icon: TrendingUp },
 ];

 const openJobs = [
   { title: "Senior Software Engineer", type: "Full-time",
salary: "$2,000 - $3,500" },
   { title: "Product Designer", type: "Full-time", salary:
"$1,500 - $2,500" },
```

```
   { title: "Data Analyst", type: "Contract", salary: "$1,200 -
$2,000" },
 ];

 const handleChange = (
   e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement |
HTMLSelectElement>
 ) => {
   setDraft((prev) => ({
     ...prev,
     [e.target.name]: e.target.value,
   }));
 };

 const openFilePicker = () => fileInputRef.current?.click();
 const openCompanyFilePicker = () =>
companyFileInputRef.current?.click();

 const handleCircleAction = () => {
   if (sourceImageUrl || saved.profileImageUrl) {
     setCropOpen(true);
     return;
   }
   openFilePicker();
 };

 const handleDeleteImage = () => {
   if (sourceImageUrl) URL.revokeObjectURL(sourceImageUrl);
   if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);

   setSourceImageUrl(null);
```

```
   setCroppedPreviewUrl(null);
   setProfileImage(null);
   setCropOpen(false);

   setSaved((prev) => ({ ...prev, profileImageUrl: "" }));
   setDraft((prev) => ({ ...prev, profileImageUrl: "" }));

   if (fileInputRef.current) fileInputRef.current.value = "";
 };

 const handleImageChange = (e:
React.ChangeEvent<HTMLInputElement>) => {
   const file = e.target.files?.[0] || null;
   if (!file) return;

   if (!["image/jpeg", "image/png",
"image/webp"].includes(file.type)) {
     alert("Only JPG, PNG, and WEBP images are allowed");
     e.target.value = "";
     return;
   }

   if (file.size > 5 * 1024 * 1024) {
     alert("Image must be 5MB or smaller");
     e.target.value = "";
     return;
   }

   if (sourceImageUrl) URL.revokeObjectURL(sourceImageUrl);
   if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);
```

```
    const objectUrl = URL.createObjectURL(file);
    setSourceImageUrl(objectUrl);
    setCroppedPreviewUrl(null);
    setProfileImage(null);
    setCropOpen(true);

    e.target.value = "";
  };

  const handleCropSave = (file: File, previewUrl: string) => {
    if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);
    setProfileImage(file);
    setCroppedPreviewUrl(previewUrl);
    setCropOpen(false);
  };

  const handleCropClose = () => setCropOpen(false);

  const handleCompanyFileChange = (e:
React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0] || null;
    if (!file) return;

    const allowedTypes = [
      "application/pdf",
      "application/msword",

"application/vnd.openxmlformats-officedocument.wordprocessingml.
document",
    ];
```

```
  if (!allowedTypes.includes(file.type)) {
    alert("Only PDF, DOC, and DOCX files are allowed");
    e.target.value = "";
    return;
  }

  if (file.size > 10 * 1024 * 1024) {
    alert("File must be 10MB or smaller");
    e.target.value = "";
    return;
  }

  setCompanyFile(file);
  setDraft((prev) => ({
    ...prev,
    companyFileName: file.name,
  }));

  e.target.value = "";
};

const handleRemoveCompanyFile = () => {
  setCompanyFile(null);
  setDraft((prev) => ({
    ...prev,
    companyFileName: "",
    companyFileUrl: "",
  }));
  setSaved((prev) => ({
    ...prev,
    companyFileName: "",
```

```
      companyFileUrl: prev.companyFileName ===
draft.companyFileName ? "" : prev.companyFileUrl,
    }));

    if (companyFileInputRef.current)
companyFileInputRef.current.value = "";
  };

  React.useEffect(() => {
    return () => {
      if (sourceImageUrl) URL.revokeObjectURL(sourceImageUrl);
      if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);
    };
  }, [sourceImageUrl, croppedPreviewUrl]);

  const handleEdit = () => {
    setDraft(saved);
    setIsEditing(true);
  };

  const handleCancel = () => {
    setDraft(saved);
    setCompanyFile(null);
    setIsEditing(false);
  };

  const handleSave = async () => {
    try {
      setLoading(true);

      const formData = new FormData();
```

```
    formData.append("companyDescription",
draft.companyDescription);
    formData.append("companyName", draft.companyName);
    formData.append("industry", draft.industry);
    formData.append("companySize", draft.companySize);
    formData.append("currentAddress", draft.currentAddress);
    formData.append("foundedYear", draft.foundedYear);
    formData.append("country", draft.country);
    formData.append("contact", draft.contact);
    formData.append("websiteLink", draft.websiteLink);

    if (profileImage) {
      formData.append("profileImage", profileImage);
    }

    if (companyFile) {
      formData.append("companyFile", companyFile);
    }

    const res = await fetch("/api/auth/employer-profile", {
      method: "POST",
      body: formData,
    });

    const text = await res.text();
    const result = text ? JSON.parse(text) : {};

    if (!res.ok) {
      alert(result.error || "Failed to update profile");
      return;
    }
```

```
      setSaved({
        ...draft,
        companyFileUrl: result.companyFileUrl ||
draft.companyFileUrl,
        companyFileName: result.companyFileName ||
draft.companyFileName,
        profileImageUrl: result.profileImageUrl ||
draft.profileImageUrl || saved.profileImageUrl,
      });
      setCompanyFile(null);
      setIsEditing(false);
    } catch (error) {
      console.error("EMPLOYER UPDATE ERROR:", error);
      alert("Something went wrong");
    } finally {
      setLoading(false);
    }
  };

  const profile = isEditing ? draft : saved;
  const displayImageSrc = croppedPreviewUrl || sourceImageUrl ||
saved.profileImageUrl || "";

  const completionFields = [
    saved.companyDescription,
    saved.companyName,
    saved.industry,
    saved.companySize,
    saved.currentAddress,
    saved.foundedYear,
    saved.country,
    saved.contact,
```

```
   saved.websiteLink,
   saved.companyFileName || saved.companyFileUrl,
   saved.profileImageUrl || croppedPreviewUrl || sourceImageUrl,
 ];
 const completion = Math.round(
   (completionFields.filter(Boolean).length /
completionFields.length) * 100
 );

 return (
   <>
     <div className="min-h-screen bg-[#edf4f1] pb-16 font-sans
text-[#10211d]">
       <header className="sticky top-0 z-50 flex items-center
justify-between bg-[#051612] px-8 py-4 text-white shadow-lg">
         <div className="flex items-center gap-2.5">
           <img src="/logo.png" alt="NexHire" className="h-8
w-8" />
           <span className="text-xl font-extrabold
tracking-tight">NexHire</span>
         </div>

         <nav className="hidden items-center gap-8 text-sm
font-semibold md:flex">
           <Link href="/dashboard"><button
className="text-gray-300
hover:text-white">Dashboard</button></Link>
           <Link href="/post_job"><button
className="text-gray-300 hover:text-white">Post
Job</button></Link>
```

```
          <Link href="/employer_message"><button
className="text-gray-300
hover:text-white">Messages</button></Link>
          <Link href="/employer_notification"><button
className="text-gray-300
hover:text-white">Notification</button></Link>
          <Link href="/employer_setting"><button
className="text-gray-300
hover:text-white">Settings</button></Link>
        </nav>

        <Link href="/employer_profile">
          <div className="group flex cursor-pointer
items-center gap-3">
            <div className="text-right">
              <p className="text-[10px] uppercase
tracking-widest text-gray-500">Company</p>
              <p className="text-sm font-bold text-white
group-hover:text-[#40b594]">Profile</p>
            </div>
            <div className="flex h-10 w-10 items-center
justify-center rounded-full bg-[#40b594] text-sm font-extrabold
text-[#051612]">
              {saved.companyName.charAt(0) || "C"}
            </div>
          </div>
        </Link>
      </header>

      <section className="relative overflow-hidden
bg-[#051612]">
        <div className="absolute inset-0 opacity-15">
```

```
          <div
            className="h-full w-full"
            style={{
              backgroundImage:
                "linear-gradient(#40b594 1px, transparent 1px),
linear-gradient(90deg, #40b594 1px, transparent 1px)",
              backgroundSize: "46px 46px",
            }}
          />
        </div>

        <div className="relative mx-auto max-w-7xl px-8 py-12
md:py-16">
          <div className="max-w-3xl">

            <h1 className="text-3xl font-black tracking-tight
text-white md:text-5xl">
              {saved.companyName}
            </h1>
            <p className="mt-4 max-w-2xl text-sm leading-7
text-white/68 md:text-base">
              Company identity, business details, hiring
summary, and stored company file in one clean layout without
overlap.
            </p>
          </div>
        </div>
      </section>

      <main className="mx-auto max-w-7xl px-8 pt-8">
        <section className={`${cardClass} mb-8 p-6 md:p-8`}>
```

```
          <div className="flex flex-col gap-8 lg:flex-row
lg:items-center lg:justify-between">
            <div className="flex flex-col items-center gap-6
text-center lg:flex-row lg:text-left">
              <div className="relative">
                <div className="flex h-36 w-36 items-center
justify-center overflow-hidden rounded-full border-[6px]
border-white bg-[linear-gradient(135deg,#21483d,#40b594)]
shadow-[0_18px_50px_rgba(0,0,0,0.18)] md:h-40 md:w-40">
                  {displayImageSrc ? (
                    <img src={displayImageSrc} alt="Company
profile" className="h-full w-full object-cover" />
                  ) : (
                    <Building2 className="h-16 w-16
text-white/85" />
                  )}
                </div>

                <button
                  type="button"
                  onClick={handleCircleAction}
                  className="absolute bottom-1 right-1 flex
h-12 w-12 items-center justify-center rounded-full border-4
border-white bg-[#00a37b] text-white shadow-lg"
                >
                  {displayImageSrc ? <Pencil size={18} /> :
<Camera size={20} />}
                </button>

                {displayImageSrc && isEditing && (
                  <button
                    type="button"
```

```
                    onClick={handleDeleteImage}
                    className="absolute left-1 top-1 flex h-10
w-10 items-center justify-center rounded-full border-4
border-white bg-white text-[#d11a2a] shadow-md"
                  >
                    <Trash2 size={16} />
                  </button>
                )}

                <input
                  ref={fileInputRef}
                  type="file"
                  accept="image/jpeg,image/png,image/webp"
                  onChange={handleImageChange}
                  className="hidden"
                />
              </div>

              <div className="max-w-2xl">
                <div className="flex items-center
justify-center gap-2 lg:justify-start">
                  <h2 className="text-2xl font-black
text-[#0d211b] md:text-3xl">
                    {saved.companyName}
                  </h2>
                  <CheckCircle2 className="text-[#40b594]"
size={20} />
                </div>
                <p className="mt-2 text-sm font-bold
text-[#40b594]">{saved.industry}</p>
                <p className="mt-4 text-sm leading-7
text-[#58706a]">
```

```
                    {saved.companyDescription}
                  </p>
                </div>
              </div>

              <div className="flex flex-wrap justify-center gap-3
lg:justify-end">
                {isEditing ? (
                  <>
                    <button
                      type="button"
                      onClick={handleCancel}
                      className="rounded-2xl border
border-[#d7e5df] bg-white px-5 py-3 text-sm font-bold
text-[#17332b]"
                    >
                      <span className="inline-flex items-center
gap-2"><X size={16} />Cancel</span>
                    </button>
                    <button
                      type="button"
                      onClick={handleSave}
                      disabled={loading}
                      className="rounded-2xl bg-[#051612] px-5
py-3 text-sm font-bold text-white disabled:opacity-60"
                    >
                      <span className="inline-flex items-center
gap-2"><Save size={16} />{loading ? "Saving..." : "Save
Changes"}</span>
                    </button>
                  </>
                ) : (
```

```
                <button
                  type="button"
                  onClick={handleEdit}
                  className="rounded-2xl bg-[#051612] px-5 py-3
text-sm font-bold text-white"
                >
                  <span className="inline-flex items-center
gap-2"><Pencil size={16} />Edit Profile</span>
                </button>
              )}
            </div>
          </div>
        </section>

        <div className="mb-8 grid grid-cols-1 gap-4
md:grid-cols-3">
          {stats.map(({ label, value, icon: Icon }) => (
            <div key={label} className={`${cardClass} p-5`}>
              <div className="mb-4 flex h-11 w-11 items-center
justify-center rounded-2xl bg-[#eff8f5] text-[#40b594]">
                <Icon size={18} />
              </div>
              <p className="text-3xl font-black
text-[#0c201a]">{value}</p>
              <p className="mt-1 text-xs font-bold uppercase
tracking-[0.22em] text-[#718881]">
                {label}
              </p>
            </div>
          ))}
        </div>
```

```
        <div className="grid grid-cols-1 gap-8
lg:grid-cols-[0.95fr_2.05fr]">
          <aside className="space-y-8">
            <section className={`${cardClass} p-6`}>
              <p className="mb-2 text-[10px] font-bold
uppercase tracking-[0.22em] text-[#40b594]">
                Profile Completion
              </p>
              <div className="mb-3 flex items-end
justify-between">
                <p className="text-3xl font-black
text-[#0c201a]">{completion}%</p>
                <p className="text-xs font-semibold
text-[#6a817b]">Complete your company profile</p>
              </div>
              <div className="h-3 w-full rounded-full
bg-[#e5efeb]">
                <div className="h-3 rounded-full bg-[#40b594]"
style={{ width: `${completion}%` }} />
              </div>
            </section>

            <section className={`${cardClass} p-6`}>
              <h3 className="mb-5 text-base font-extrabold
text-[#071a15]">Quick Info</h3>
              <div className="space-y-4">
                <InfoRow icon={Globe} label="Website"
value={saved.websiteLink || "Not added"} />
                <InfoRow icon={MapPin} label="Location"
value={saved.currentAddress || "Not added"} />
                <InfoRow icon={Users} label="Company Size"
value={saved.companySize || "Not added"} />
```

```
                  <InfoRow icon={Phone} label="Contact"
value={saved.contact || "Not added"} />
                </div>
              </section>

              <section className={`${cardClass} p-6`}>
                <div className="mb-5 flex items-center gap-3">
                  <div className="flex h-11 w-11 items-center
justify-center rounded-2xl bg-[#eff8f5] text-[#40b594]">
                    <FileText size={18} />
                  </div>
                  <div>
                    <h3 className="text-base font-extrabold
text-[#071a15]">Company File</h3>
                    <p className="text-xs text-[#6a817b]">Store
one company document</p>
                  </div>
                </div>

                <div className="rounded-3xl border border-dashed
border-[#cfe2db] bg-[#f8fcfa] p-4">
                  <p className="text-sm font-semibold
text-[#17332b]">
                    {saved.companyFileName || "No company file
uploaded yet"}
                  </p>
                  {saved.companyFileUrl ? (
                    <a
                      href={saved.companyFileUrl}
                      target="_blank"
                      rel="noreferrer"
```

```
                    className="mt-3 inline-flex items-center
gap-2 text-sm font-bold text-[#0a7e61]"
                  >
                    <FileText size={16} />
                    View uploaded file
                  </a>
                ) : (
                  <p className="mt-2 text-xs
text-[#7a8f89]">PDF, DOC, or DOCX</p>
                )}
              </div>
            </section>
          </aside>

          <div className="space-y-8">
            <section className={`${cardClass} p-3`}>
              <div className="flex flex-wrap gap-2">
                {(["overview", "details"] as const).map((tab)
=> (
                  <button
                    key={tab}
                    type="button"
                    onClick={() => setActiveTab(tab)}
                    className={`rounded-2xl px-5 py-3 text-sm
font-extrabold ${
                      activeTab === tab ? "bg-[#051612]
text-white" : "text-[#5f7770]"
                    }`}
                  >
                    {tab === "overview" ? "Overview" : "Company
Details"}
                  </button>
```

```
                ))}
              </div>
            </section>

            {activeTab === "overview" ? (
              <>
                <section className={`${cardClass} p-6 md:p-8`}>
                  <h3 className="mb-6 text-xl font-black
text-[#0d211b]">About Company</h3>
                  {isEditing ? (
                    <textarea
                      name="companyDescription"
                      value={profile.companyDescription}
                      onChange={handleChange}
                      className="h-40 w-full resize-none
rounded-3xl border border-[#d9e8e2] bg-[#f6fbf8] px-5 py-4
text-sm"
                    />
                  ) : (
                    <p className="text-sm leading-8
text-[#405752]">{saved.companyDescription}</p>
                  )}
                </section>

                <section className={`${cardClass} p-6 md:p-8`}>
                  <div className="mb-6 flex items-center
justify-between">
                    <h3 className="text-xl font-black
text-[#0d211b]">Current Open Positions</h3>
                  </div>

                  <div className="space-y-4">
```

```
                    {openJobs.map((job) => (
                      <div
                        key={job.title}
                        className="flex items-center
justify-between rounded-3xl border border-[#ebf1ee] bg-[#f9fcfb]
px-5 py-4"
                      >
                        <div className="flex items-center
gap-4">
                          <div className="flex h-11 w-11
items-center justify-center rounded-2xl bg-[#eff8f5]
text-[#16322b]">
                            <Briefcase size={18} />
                          </div>
                          <div>
                            <h4 className="text-sm font-black
text-[#10211d]">{job.title}</h4>
                            <p className="mt-1 text-xs
font-semibold text-[#59706a]">
                              {job.type} · <span
className="text-[#40b594]">{job.salary}</span>
                            </p>
                          </div>
                        </div>
                        <span className="rounded-full bg-white
px-3 py-1 text-[11px] font-bold uppercase tracking-[0.18em]
text-[#59706a]">
                          Active
                        </span>
                      </div>
                    ))}
                  </div>
```

```
                  </section>
                </>
              ) : (
                <section className={`${cardClass} p-6 md:p-8`}>
                  <h3 className="mb-6 text-xl font-black
text-[#0d211b]">Company Information</h3>
                  <div className="grid grid-cols-1 gap-5
md:grid-cols-2">
                    <EditableField editing={isEditing}
label="Company Name" name="companyName"
value={profile.companyName} onChange={handleChange}
className="md:col-span-2" />
                    <EditableField editing={isEditing}
label="Industry" name="industry" value={profile.industry}
onChange={handleChange} />
                    <EditableField editing={isEditing}
label="Company Size" name="companySize"
value={profile.companySize} onChange={handleChange} />
                    <EditableField editing={isEditing}
label="Current Address" name="currentAddress"
value={profile.currentAddress} onChange={handleChange}
className="md:col-span-2" />
                    <EditableSelectYear editing={isEditing}
label="Founded Year" name="foundedYear"
value={profile.foundedYear} onChange={handleChange} />
                    <EditableField editing={isEditing}
label="Country" name="country" value={profile.country}
onChange={handleChange} />
                    <EditableField editing={isEditing}
label="Contact" name="contact" value={profile.contact}
onChange={handleChange} />
```

```
                    <EditableField editing={isEditing}
label="Website Link" name="websiteLink"
value={profile.websiteLink} onChange={handleChange} />
                  </div>
                </section>
              )}

              <section className={`${cardClass} p-6 md:p-8`}>
                <div className="mb-6 flex items-center
justify-between">
                  <div>
                    <h3 className="text-xl font-black
text-[#0d211b]">Upload Company File</h3>
                    <p className="mt-1 text-sm text-[#6a817b]">
                      Store your company profile, brochure, or
supporting document
                    </p>
                  </div>
                  <FileText className="text-[#40b594]" size={22}
/>
                </div>

                <div className="rounded-[28px] border
border-dashed border-[#cfe2db] bg-[#f8fcfa] p-6">
                  <div className="flex flex-col gap-4 md:flex-row
md:items-center md:justify-between">
                    <div>
                      <p className="text-sm font-bold
text-[#17332b]">
                        {profile.companyFileName ||
saved.companyFileName || "No file selected"}
                      </p>
```

```
                    <p className="mt-1 text-xs text-[#7a8f89]">
                      Accepted formats: PDF, DOC, DOCX. Max
size: 10MB.
                    </p>
                  </div>

                  <div className="flex flex-wrap gap-3">
                    {isEditing && (
                      <>
                        <button
                          type="button"
                          onClick={openCompanyFilePicker}
                          className="inline-flex items-center
gap-2 rounded-2xl bg-[#051612] px-4 py-3 text-sm font-bold
text-white"
                        >
                          <Upload size={16} />
                          Upload File
                        </button>

                        {(profile.companyFileName ||
saved.companyFileName) && (
                          <button
                            type="button"
                            onClick={handleRemoveCompanyFile}
                            className="inline-flex items-center
gap-2 rounded-2xl border border-[#d7e5df] bg-white px-4 py-3
text-sm font-bold text-[#b42318]"
                          >
                            <Trash2 size={16} />
                            Remove File
                          </button>
```

```
                        )}
                      </>
                    )}

                    {!isEditing && saved.companyFileUrl && (
                      <a
                        href={saved.companyFileUrl}
                        target="_blank"
                        rel="noreferrer"
                        className="inline-flex items-center
gap-2 rounded-2xl bg-[#eff8f5] px-4 py-3 text-sm font-bold
text-[#0a7e61]"
                      >
                        <FileText size={16} />
                        View File
                      </a>
                    )}
                  </div>
                </div>

                <input
                  ref={companyFileInputRef}
                  type="file"

accept=".pdf,.doc,.docx,application/pdf,application/msword,appli
cation/vnd.openxmlformats-officedocument.wordprocessingml.docume
nt"
                  onChange={handleCompanyFileChange}
                  className="hidden"
                />
              </div>
            </section>
```

```
            </div>
          </div>
        </main>
      </div>

      <ImageCropModal
        open={cropOpen}
        imageSrc={sourceImageUrl || saved.profileImageUrl}
        title="Adjust company profile image"
        onClose={handleCropClose}
        onSave={handleCropSave}
        onUploadAnother={openFilePicker}
        onDelete={handleDeleteImage}
        hasImage={!!(sourceImageUrl || saved.profileImageUrl)}
      />
    </>
  );
}

function EditableField({
  editing,
  label,
  name,
  value,
  onChange,
  className = "",
}: {
  editing: boolean;
  label: string;
  name: string;
  value: string;
  onChange: (
```

```
   e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement |
HTMLSelectElement>
 ) => void;
 className?: string;
}) {
 return (
   <div className={className}>
     <p className="mb-2 text-sm font-bold
text-[#27413a]">{label}</p>
     {editing ? (
       <input
         name={name}
         value={value}
         onChange={onChange}
         className="w-full rounded-2xl border border-[#d9e8e2]
bg-[#f6fbf8] px-4 py-3 text-sm"
       />
     ) : (
       <div className="rounded-2xl border border-[#edf3f0]
bg-[#f9fcfb] px-4 py-3 text-sm font-semibold">
         {value || "Not added"}
       </div>
     )}
   </div>
 );
}

function EditableSelectYear({
 editing,
 label,
 name,
 value,
```

```
 onChange,
}: {
 editing: boolean;
 label: string;
 name: string;
 value: string;
 onChange: (
   e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement |
HTMLSelectElement>
 ) => void;
}) {
 return (
   <div>
     <p className="mb-2 text-sm font-bold
text-[#27413a]">{label}</p>
     {editing ? (
       <select
         name={name}
         value={value}
         onChange={onChange}
         className="w-full rounded-2xl border border-[#d9e8e2]
bg-[#f6fbf8] px-4 py-3 text-sm"
       >
         <option value="">Select option</option>
         {Array.from({ length: 60 }, (_, i) => {
           const year = new Date().getFullYear() - i;
           return (
             <option key={year} value={String(year)}>
               {year}
             </option>
           );
         })}
```

```
        </select>
      ) : (
        <div className="rounded-2xl border border-[#edf3f0]
bg-[#f9fcfb] px-4 py-3 text-sm font-semibold">
          {value || "Not added"}
        </div>
      )}
    </div>
  );
}

function InfoRow({
  icon: Icon,
  label,
  value,
}: {
  icon: React.ComponentType<{ size?: number }>;
  label: string;
  value: string;
}) {
  return (
    <div className="flex items-center gap-4">
      <div className="flex h-10 w-10 items-center justify-center
rounded-xl bg-[#f0f4f3] text-[#40b594]">
        <Icon size={17} />
      </div>
      <div>
        <p className="text-[10px] font-bold uppercase
tracking-[0.22em] text-[#6b7f79]">
          {label}
        </p>
```

```
        <p className="text-sm font-bold
text-[#071a15]">{value}</p>
      </div>
    </div>
  );
}

"use client";

import React, { useState } from "react";
import {
  Shield,
  AlertTriangle,
  Mail,
  Save,
  Building2,
  Globe,
  MapPin,
  Trash2,
  LogOut,
  ChevronRight,
  Lock,
  Eye,
  EyeOff,
  Briefcase,
  ExternalLink,
  UserCog,
} from "lucide-react";
import Link from "next/link";
import { useRouter } from "next/navigation";

const EmployerSettings = () => {
```

```
 const router = useRouter();
 const [activeTab, setActiveTab] = useState("general");
 const [showCurrent, setShowCurrent] = useState(false);
 const [showNew, setShowNew] = useState(false);
 const [deleteText, setDeleteText] = useState("");
 const [savingEmail, setSavingEmail] = useState(false);
 const [savingPassword, setSavingPassword] = useState(false);
 const [currentPassword, setCurrentPassword] = useState("");
 const [newPassword, setNewPassword] = useState("");

 const [settings, setSettings] = useState({
   adminEmail: "",
   billingEmail: "",
 });

 const tabs = [
   { id: "general", name: "General", icon: UserCog },
   { id: "security", name: "Security", icon: Shield },
   { id: "danger", name: "Danger Zone", icon: AlertTriangle },
 ];

 const inputClass =
   "w-full rounded-xl border border-gray-200 bg-[#f8faf9] px-4
py-3 text-sm font-medium text-[#071a15] placeholder-[#9ab0aa]
transition-all focus:border-[#40b594] focus:outline-none
focus:ring-2 focus:ring-[#40b594]/30";

 const labelClass = "mb-2 block text-sm font-extrabold
text-[#071a15]";

 const handleSignOut = async () => {
   await fetch("/api/auth/logout", { method: "POST" });
```

```
  router.push("/login");
};

const handleSaveEmails = async () => {
  setSavingEmail(true);
  await new Promise((r) => setTimeout(r, 500));
  setSavingEmail(false);
  alert("Email settings saved");
};

const handleUpdatePassword = async () => {
  if (!currentPassword || !newPassword) {
    alert("Please fill in both password fields");
    return;
  }
  if (newPassword.length < 8) {
    alert("New password must be at least 8 characters");
    return;
  }
  setSavingPassword(true);
  const res = await fetch("/api/auth/reset-password", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ password: newPassword,
confirmPassword: newPassword }),
  });
  setSavingPassword(false);
  if (res.ok) {
    alert("Password updated successfully");
    setCurrentPassword("");
    setNewPassword("");
  } else {
```

```
    alert("Failed to update password");
  }
};

const handleDeleteAccount = async () => {
  if (deleteText !== "DELETE") return;
  const res = await fetch("/api/auth/logout", { method: "POST"
});
  alert("Account deletion requested. Contact support.");
  router.push("/login");
};

return (
  <div className="min-h-screen bg-[#f0f4f3] pb-16 font-sans">
    <header className="sticky top-0 z-50 flex items-center
justify-between bg-[#051612] px-8 py-4 text-white shadow-lg">
      <div className="flex items-center gap-2.5">
        <img src="/logo.png" alt="NexHire" className="h-8 w-8"
/>
        <span className="text-xl font-extrabold
tracking-tight">NexHire</span>
      </div>

      <nav className="hidden items-center gap-8 text-sm
font-semibold md:flex">
        <Link href="/dashboard">
          <button className="text-gray-300 transition-colors
hover:text-white">Dashboard</button>
        </Link>
        <Link href="/post_job">
          <button className="text-gray-300 transition-colors
hover:text-white">Post Job</button>
```

```
        </Link>
        <Link href="/employer_message">
          <button className="text-gray-300 transition-colors
hover:text-white">Messages</button>
        </Link>
        <Link href="/employer_notification">
          <button className="text-gray-300 transition-colors
hover:text-white">Notification</button>
        </Link>
        <Link href="/subscription">
          <button className="text-gray-300 transition-colors
hover:text-white">Subscription</button>
        </Link>
        <button className="border-b-2 border-[#40b594] pb-1
text-[#40b594]">Settings</button>
      </nav>

      <Link href="/employer_profile">
        <div className="group flex cursor-pointer items-center
gap-3">
          <div className="text-right">
            <p className="text-[10px] uppercase tracking-widest
text-gray-500">Company</p>
            <p className="text-sm font-bold text-white
transition-colors group-hover:text-[#40b594]">
              Profile
            </p>
          </div>
          <div className="flex h-10 w-10 items-center
justify-center rounded-full bg-[#40b594] text-sm font-extrabold
text-[#051612]">
            C
```

```
            </div>
          </div>
        </Link>
      </header>

      <main className="mx-auto max-w-6xl px-6 py-10 md:px-10">
        <div className="mb-8">
          <p className="mb-1 text-xs font-bold uppercase
tracking-widest text-[#40b594]">Account</p>
          <h1 className="text-4xl font-extrabold
text-[#071a15]">Settings</h1>
          <p className="mt-1 font-medium text-[#4a5a55]">
            Manage account controls, security, and workspace
preferences
          </p>
        </div>

        <div className="flex flex-col gap-6 lg:flex-row">
          <aside className="flex-shrink-0 lg:w-72">
            <div className="overflow-hidden rounded-2xl border
border-gray-100 bg-white shadow-sm">
              {tabs.map((tab, idx) => {
                const Icon = tab.icon;
                const isActive = activeTab === tab.id;
                const isDanger = tab.id === "danger";

                return (
                  <button
                    key={tab.id}
                    onClick={() => setActiveTab(tab.id)}
                    className={`w-full px-5 py-4 text-sm
font-bold transition-all ${
```

```
                    idx !== 0 ? "border-t border-gray-100" : ""
                  } ${
                    isActive
                      ? isDanger
                        ? "bg-red-50 text-red-600"
                        : "bg-[#f0f9f6] text-[#071a15]"
                      : isDanger
                      ? "text-red-400 hover:bg-red-50
hover:text-red-600"
                      : "text-[#4a5a55] hover:bg-[#f8faf9]
hover:text-[#071a15]"
                  }`}
                >
                  <div className="flex items-center
justify-between">
                    <div className="flex items-center gap-3">
                      <div
                        className={`flex h-8 w-8 items-center
justify-center rounded-lg ${
                          isActive
                            ? isDanger
                              ? "bg-red-100"
                              : "bg-[#d1e8e3]"
                            : "bg-[#f0f4f3]"
                        }`}
                      >
                        <Icon
                          size={16}
                          className={
                            isActive
                              ? isDanger
                                ? "text-red-600"
```

```
                                  : "text-[#40b594]"
                                : isDanger
                                ? "text-red-400"
                                : "text-[#6b7f79]"
                            }
                          />
                        </div>
                        {tab.name}
                      </div>
                      {isActive && (
                        <ChevronRight
                          size={16}
                          className={isDanger ? "text-red-400" :
"text-[#40b594]"}
                        />
                      )}
                    </div>
                  </button>
                );
              })}
            </div>

            <button
              onClick={handleSignOut}
              className="mt-4 flex w-full items-center gap-3
rounded-2xl border border-gray-100 bg-white px-5 py-4 text-sm
font-bold text-[#4a5a55] shadow-sm transition-all
hover:border-red-200 hover:bg-red-50 hover:text-red-600"
            >
              <div className="flex h-8 w-8 items-center
justify-center rounded-lg bg-[#f0f4f3]">
                <LogOut size={16} className="text-[#6b7f79]" />
```

```
            </div>
            Sign Out
          </button>
        </aside>

        <div className="min-w-0 flex-1">
          {activeTab === "general" && (
            <div className="space-y-6">
              <div className="overflow-hidden rounded-2xl
border border-gray-100 bg-white shadow-sm">
                <div className="border-b border-gray-100 px-8
py-6">
                  <h2 className="text-lg font-extrabold
text-[#071a15]">Workspace Overview</h2>
                  <p className="mt-0.5 text-sm font-medium
text-[#4a5a55]">
                    Company information is managed from the
public employer profile page
                  </p>
                </div>

                <div className="p-8">
                  <div className="mt-6 rounded-2xl border
border-[#d1e8e3] bg-[#f0f9f6] p-5">
                    <div className="flex flex-col gap-4
md:flex-row md:items-center md:justify-between">
                      <div>
                        <p className="text-sm font-extrabold
text-[#071a15]">
                          Edit company profile from the real
profile page
                        </p>
```

```
                        <p className="mt-1 text-sm font-medium
text-[#4a5a55]">
                          Keep branding, company details,
photo, and files in one place.
                        </p>
                      </div>
                      <Link href="/employer_profile">
                        <button className="inline-flex
items-center gap-2 rounded-xl bg-[#051612] px-5 py-3 text-sm
font-extrabold text-white transition-all hover:bg-[#0d2a23]">
                          Open Profile
                          <ExternalLink size={16} />
                        </button>
                      </Link>
                    </div>
                  </div>
                </div>
              </div>

              <div className="overflow-hidden rounded-2xl
border border-gray-100 bg-white shadow-sm">
                <div className="border-b border-gray-100 px-8
py-6">
                  <h2 className="text-lg font-extrabold
text-[#071a15]">Account Emails</h2>
                </div>

                <div className="p-8">
                  <form
                    className="space-y-6"
                    onSubmit={(e) => {
                      e.preventDefault();
```

```
                        handleSaveEmails();
                      }}
                    >
                      <div>
                        <label className={labelClass}>Admin
Email</label>
                        <div className="relative">
                          <Mail size={16} className="absolute
left-4 top-1/2 -translate-y-1/2 text-[#6b7f79]" />
                          <input
                            type="email"
                            value={settings.adminEmail}
                            onChange={(e) =>
                              setSettings((prev) => ({ ...prev,
adminEmail: e.target.value }))
                            }
                            className={`${inputClass} pl-10`}
                          />
                        </div>
                      </div>

                      <div>
                        <label className={labelClass}>Billing
Email</label>
                        <div className="relative">
                          <Mail size={16} className="absolute
left-4 top-1/2 -translate-y-1/2 text-[#6b7f79]" />
                          <input
                            type="email"
                            value={settings.billingEmail}
                            onChange={(e) =>
```

```
                            setSettings((prev) => ({ ...prev,
billingEmail: e.target.value }))
                          }
                          className={`${inputClass} pl-10`}
                        />
                      </div>
                    </div>

                    <div className="pt-2">
                      <button
                        type="submit"
                        disabled={savingEmail}
                        className="flex items-center gap-2
rounded-xl bg-[#051612] px-6 py-3 text-sm font-extrabold
text-white transition-all hover:bg-[#0d2a23]
disabled:opacity-60"
                      >
                        <Save size={16} />
                        {savingEmail ? "Saving..." : "Save
Changes"}
                      </button>
                    </div>
                  </form>
                </div>
              </div>
            </div>
          )}

          {activeTab === "security" && (
            <div className="overflow-hidden rounded-2xl border
border-gray-100 bg-white shadow-sm">
```

```
                <div className="border-b border-gray-100 px-8
py-6">
                  <h2 className="text-lg font-extrabold
text-[#071a15]">Security</h2>
                  <p className="mt-0.5 text-sm font-medium
text-[#4a5a55]">
                    Update your account password
                  </p>
                </div>

                <div className="p-8">
                  <form
                    className="max-w-md space-y-6"
                    onSubmit={(e) => {
                      e.preventDefault();
                      handleUpdatePassword();
                    }}
                  >
                    <div>
                      <label className={labelClass}>Current
Password</label>
                      <div className="relative">
                        <Lock size={16} className="absolute
left-4 top-1/2 -translate-y-1/2 text-[#6b7f79]" />
                        <input
                          type={showCurrent ? "text" :
"password"}
                          value={currentPassword}
                          onChange={(e) =>
setCurrentPassword(e.target.value)}
                          placeholder="Enter current password"
                          className={`${inputClass} pl-10 pr-10`}
```

```
                        />
                        <button
                          type="button"
                          onClick={() =>
setShowCurrent(!showCurrent)}
                          className="absolute right-4 top-1/2
-translate-y-1/2 text-[#6b7f79] transition-colors
hover:text-[#071a15]"
                        >
                          {showCurrent ? <EyeOff size={16} /> :
<Eye size={16} />}
                        </button>
                      </div>
                    </div>

                    <div>
                      <label className={labelClass}>New
Password</label>
                      <div className="relative">
                        <Lock size={16} className="absolute
left-4 top-1/2 -translate-y-1/2 text-[#6b7f79]" />
                        <input
                          type={showNew ? "text" : "password"}
                          value={newPassword}
                          onChange={(e) =>
setNewPassword(e.target.value)}
                          placeholder="Enter new password"
                          className={`${inputClass} pl-10 pr-10`}
                        />
                        <button
                          type="button"
                          onClick={() => setShowNew(!showNew)}
```

```
                        className="absolute right-4 top-1/2
-translate-y-1/2 text-[#6b7f79] transition-colors
hover:text-[#071a15]"
                      >
                        {showNew ? <EyeOff size={16} /> : <Eye
size={16} />}
                      </button>
                    </div>
                  </div>

                  <div className="rounded-xl border
border-[#d1e8e3] bg-[#f0f9f6] p-4">
                    <p className="mb-2 text-xs font-extrabold
text-[#071a15]">Password requirements</p>
                    {["At least 8 characters", "One uppercase
letter", "One number or symbol"].map((req) => (
                      <div key={req} className="mt-1.5 flex
items-center gap-2">
                        <div className="h-1.5 w-1.5
rounded-full bg-[#40b594]" />
                        <p className="text-xs font-medium
text-[#4a5a55]">{req}</p>
                      </div>
                    ))}
                  </div>

                  <div className="pt-2">
                    <button
                      type="submit"
                      disabled={savingPassword}
                      className="flex items-center gap-2
rounded-xl bg-[#051612] px-6 py-3 text-sm font-extrabold
```

```
text-white transition-all hover:bg-[#0d2a23]
disabled:opacity-60"
                    >
                      <Shield size={16} />
                      {savingPassword ? "Updating..." : "Update
Password"}
                    </button>
                  </div>
                </form>
              </div>
            </div>
          )}

          {activeTab === "danger" && (
            <div className="overflow-hidden rounded-2xl border
border-red-100 bg-white shadow-sm">
              <div className="border-b border-red-100
bg-red-50/50 px-8 py-6">
                <h2 className="text-lg font-extrabold
text-red-600">Danger Zone</h2>
                <p className="mt-0.5 text-sm font-medium
text-red-400">
                  These actions are permanent and cannot be
undone
                </p>
              </div>

              <div className="p-8">
                <div className="rounded-2xl border
border-red-100 bg-red-50/30 p-7">
                  <div className="mb-6 flex items-start gap-4">
```

```
                    <div className="flex h-10 w-10
flex-shrink-0 items-center justify-center rounded-xl
bg-red-100">
                      <Trash2 size={18}
className="text-red-600" />
                    </div>
                    <div>
                      <h3 className="text-base font-extrabold
text-[#071a15]">
                        Delete Company Workspace
                      </h3>
                      <p className="mt-1 text-sm font-medium
leading-relaxed text-[#4a5a55]">
                        This will permanently delete your
company profile, all active job listings,
                        and applicant history. This action
cannot be reversed.
                      </p>
                    </div>
                  </div>

                  <div className="space-y-4">
                    <div>
                      <label className="mb-2 block text-sm
font-extrabold text-[#071a15]">
                        Type <span className="font-black
tracking-widest text-red-600">DELETE</span> to confirm
                      </label>
                      <input
                        type="text"
                        placeholder="Type DELETE here"
                        value={deleteText}
```

```
                        onChange={(e) =>
setDeleteText(e.target.value)}
                        className="w-full max-w-sm rounded-xl
border border-red-200 bg-white px-4 py-3 text-sm font-medium
text-[#071a15] placeholder-red-200 transition-all
focus:border-red-400 focus:outline-none focus:ring-2
focus:ring-red-300/40"
                      />
                    </div>

                    <button
                      type="button"
                      onClick={handleDeleteAccount}
                      disabled={deleteText !== "DELETE"}
                      className={`flex items-center gap-2
rounded-xl px-6 py-3 text-sm font-extrabold transition-all ${
                        deleteText === "DELETE"
                          ? "bg-red-600 text-white
hover:bg-red-700"
                          : "cursor-not-allowed bg-red-100
text-red-300"
                      }`}
                    >
                      <Trash2 size={16} />
                      Delete Company Account
                    </button>
                  </div>
                </div>
              </div>
            </div>
          )}
        </div>
```

```
        </div>
      </main>
    </div>
  );
};

function SummaryCard({
  icon: Icon,
  label,
  value,
}: {
  icon: React.ComponentType<{ size?: number; className?: string
}>;
  label: string;
  value: string;
}) {
  return (
    <div className="rounded-2xl border border-gray-100
bg-[#f8faf9] p-5">
      <div className="mb-3 flex h-10 w-10 items-center
justify-center rounded-xl bg-[#eaf5f1]">
        <Icon size={18} className="text-[#40b594]" />
      </div>
      <p className="text-[11px] font-black uppercase
tracking-wider text-[#6b7f79]">{label}</p>
      <p className="mt-1 text-sm font-extrabold
text-[#071a15]">{value}</p>
    </div>
  );
}

export default EmployerSettings;
```

```
"use client";

import React, { useState } from "react";
import {
 Clock,
 FileEdit,
 Trash2,
 Briefcase,
 PlusCircle,
 ChevronDown,
 CheckCircle,
 X,
 AlertCircle,
} from "lucide-react";
import Link from "next/link";
import EmployerNavProfile from
"@/components/ui/EmployerNavProfile";

const categoryOptions = [
 "IT & Software",
 "Design",
 "Marketing",
 "Sales",
 "Finance",
 "Human Resources",
 "Engineering",
 "Customer Support",
 "Operations",
 "Legal",
];

const locationOptions = [
```

```
 "Phnom Penh",
 "Siem Reap",
 "Battambang",
 "Sihanoukville",
 "Kampot",
 "Kandal",
 "Takeo",
 "Remote",
 "Other",
];

type ToastType = "success" | "error";

type Toast = {
 id: number;
 type: ToastType;
 message: string;
};

function useToast() {
 const [toasts, setToasts] = useState<Toast[]>([]);

 const show = (message: string, type: ToastType = "success") =>
{
   const id = Date.now();
   setToasts((prev) => [...prev, { id, type, message }]);
   setTimeout(() => {
     setToasts((prev) => prev.filter((t) => t.id !== id));
   }, 4000);
 };

 const remove = (id: number) => {
```

```
    setToasts((prev) => prev.filter((t) => t.id !== id));
  };

  return { toasts, show, remove };
}

function ToastContainer({
  toasts,
  onRemove,
}: {
  toasts: Toast[];
  onRemove: (id: number) => void;
}) {
  if (toasts.length === 0) return null;
  return (
    <div className="fixed bottom-6 right-6 z-50 flex flex-col
gap-3">
      {toasts.map((toast) => (
        <div
          key={toast.id}
          className={`flex items-center gap-3 rounded-2xl px-5
py-4 shadow-lg border min-w-[280px] max-w-sm ${
            toast.type === "success"
              ? "bg-white border-green-200"
              : "bg-white border-red-200"
          }`}
        >
          {toast.type === "success" ? (
            <CheckCircle size={18} className="text-green-500
shrink-0" />
          ) : (
```

```
            <AlertCircle size={18} className="text-red-500
shrink-0" />
          )}
          <p
            className={`text-sm font-bold flex-1 ${
              toast.type === "success" ? "text-green-700" :
"text-red-700"
            }`}
          >
            {toast.message}
          </p>
          <button
            onClick={() => onRemove(toast.id)}
            className="text-gray-400 hover:text-gray-600
shrink-0"
          >
            <X size={15} />
          </button>
        </div>
      ))}
    </div>
  );
}

function Combobox({
  placeholder,
  options,
  inputClass,
  value,
  onChange,
}: {
  placeholder: string;
```

```
 options: string[];
 inputClass: string;
 value: string;
 onChange: (value: string) => void;
}) {
 const [open, setOpen] = useState(false);

 const filtered = options.filter((o) =>
   o.toLowerCase().includes(value.toLowerCase())
 );

 return (
   <div className="relative">
     <input
       type="text"
       value={value}
       placeholder={placeholder}
       className={`${inputClass} pr-10`}
       onChange={(e) => {
         onChange(e.target.value);
         setOpen(true);
       }}
       onFocus={() => setOpen(true)}
       onBlur={() => setTimeout(() => setOpen(false), 120)}
     />
     <ChevronDown
       size={16}
       className={`absolute right-4 top-1/2 -translate-y-1/2
text-[#6b7f79] pointer-events-none transition-transform
duration-200 ${
         open ? "rotate-180" : ""
       }`}
```

```
      />
      {open && filtered.length > 0 && (
        <ul className="absolute z-50 mt-1.5 w-full bg-white
border border-gray-200 rounded-xl shadow-lg overflow-hidden">
          {filtered.map((option) => (
            <li
              key={option}
              onMouseDown={() => {
                onChange(option);
                setOpen(false);
              }}
              className={`px-4 py-2.5 text-sm font-medium
text-[#071a15] cursor-pointer transition-colors
hover:bg-[#f0f9f6] hover:text-[#40b594] ${
                value === option ? "bg-[#f0f9f6] text-[#40b594]
font-bold" : ""
              }`}
            >
              {option}
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}

type DraftJob = {
  id: string;
  title: string;
  category: string;
  lastSaved: string;
```

```
};

const emptyForm = {
 title: "",
 category: "",
 location: "",
 arrangement: "",
 employmentType: "",
 experienceLevel: "",
 salaryMin: "",
 salaryMax: "",
 applicationDeadline: "",
 description: "",
 requirements: "",
 applicationPlatform: "",
 externalApplyLink: "",
 contactEmail: "",
 status: "active",
};

const PostJob = () => {
 const { toasts, show, remove } = useToast();
 const [activeTab, setActiveTab] = useState<"post" |
"drafts">("post");
 const [loading, setLoading] = useState(false);
 const [draftLoading, setDraftLoading] = useState(false);
 const [draftData, setDraftData] = useState<DraftJob[]>([]);
 const [deletingId, setDeletingId] = useState<string |
null>(null);
 const [form, setForm] = useState(emptyForm);

 const inputClass =
```

```
   "w-full px-4 py-3 border border-gray-200 rounded-xl
bg-[#f8faf9] text-[#071a15] text-sm font-medium
placeholder-[#9ab0aa] focus:outline-none focus:ring-2
focus:ring-[#40b594]/30 focus:border-[#40b594] transition-all";

 const selectClass =
   "w-full px-4 py-3 border border-gray-200 rounded-xl
bg-[#f8faf9] text-[#071a15] text-sm font-medium
focus:outline-none focus:ring-2 focus:ring-[#40b594]/30
focus:border-[#40b594] transition-all appearance-none";

 const labelClass = "block text-sm font-extrabold text-[#071a15]
mb-2";

 const arrangementValue =
   form.arrangement === "On-site"
     ? "on_site"
     : form.arrangement === "Remote"
     ? "remote"
     : form.arrangement === "Hybrid"
     ? "hybrid"
     : "on_site";

 const employmentValue =
   form.employmentType === "Full-time"
     ? "full_time"
     : form.employmentType === "Part-time"
     ? "part_time"
     : form.employmentType === "Contract"
     ? "contract"
     : form.employmentType === "Freelance"
     ? "freelance"
```

```
    : form.employmentType === "Internship"
    ? "internship"
    : "full_time";

const experienceValue =
  form.experienceLevel === "Entry Level"
    ? "entry"
    : form.experienceLevel === "Mid Level"
    ? "mid"
    : form.experienceLevel === "Senior"
    ? "senior"
    : form.experienceLevel === "Lead / Manager"
    ? "lead"
    : form.experienceLevel === "Executive"
    ? "executive"
    : "entry";

const handleSubmit = async (status: "active" | "draft") => {
  if (!form.title.trim()) {
    show("Job title is required.", "error");
    return;
  }
  if (!form.category.trim()) {
    show("Job category is required.", "error");
    return;
  }
  if (!form.location.trim()) {
    show("Location is required.", "error");
    return;
  }
  if (!form.description.trim()) {
    show("Job description is required.", "error");
```

```
      return;
    }

    try {
      setLoading(true);

      const res = await fetch("/api/jobs", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          title: form.title,
          category: form.category,
          location: form.location,
          arrangement: arrangementValue,
          employmentType: employmentValue,
          experienceLevel: experienceValue,
          salaryMin: form.salaryMin ? Number(form.salaryMin) :
null,
          salaryMax: form.salaryMax ? Number(form.salaryMax) :
null,
          applicationDeadline: form.applicationDeadline || null,
          description: form.description,
          requirements: form.requirements,
          applicationPlatform: form.applicationPlatform ||
"internal",
          externalApplyLink: form.externalApplyLink || null,
          contactEmail: form.contactEmail || null,
          status,
        }),
      });

      const data = await res.json();
```

```
    if (!res.ok) {
      show(data.error ?? "Failed to save job. Please try
again.", "error");
      return;
    }

    show(
      status === "active"
        ? "Job published successfully!"
        : "Draft saved successfully!",
      "success"
    );

    setForm(emptyForm);

    if (status === "draft") {
      await loadDrafts();
      setActiveTab("drafts");
    }
  } catch {
    show("Something went wrong. Please try again.", "error");
  } finally {
    setLoading(false);
  }
};

const loadDrafts = async () => {
  try {
    setDraftLoading(true);
    const res = await fetch("/api/jobs?mine=1", { cache:
"no-store" });
```

```
      const data = await res.json();
      if (!res.ok) return;

      const drafts = (data || [])
        .filter((job: { status: string }) => job.status ===
"draft")
        .map((job: { id: string; title: string; category: string;
postedAt: string; createdAt: string }) => ({
          id: job.id,
          title: job.title,
          category: job.category,
          lastSaved: new Date(job.postedAt ??
job.createdAt).toLocaleString(),
        }));

      setDraftData(drafts);
    } catch (err) {
      console.error(err);
    } finally {
      setDraftLoading(false);
    }
  };

  const handleDeleteDraft = async (jobId: string) => {
    try {
      setDeletingId(jobId);
      const res = await fetch(`/api/jobs/${jobId}`, { method:
"DELETE" });
      if (res.ok) {
        setDraftData((prev) => prev.filter((d) => d.id !==
jobId));
        show("Draft deleted.", "success");
```

```
      } else {
        show("Failed to delete draft.", "error");
      }
    } catch {
      show("Something went wrong.", "error");
    } finally {
      setDeletingId(null);
    }
  };

  React.useEffect(() => {
    if (activeTab === "drafts") loadDrafts();
  }, [activeTab]);

  return (
    <div className="min-h-screen font-sans pb-16" style={{
background: "#f0f4f3" }}>

      <ToastContainer toasts={toasts} onRemove={remove} />

      <header className="bg-[#051612] text-white px-8 py-4 flex
items-center justify-between sticky top-0 z-50 shadow-lg">
        <div className="flex items-center gap-2.5">
          <img src="/logo.png" alt="NexHire" className="w-8 h-8"
/>
          <span className="text-xl font-extrabold
tracking-tight">NexHire</span>
        </div>
        <nav className="hidden md:flex items-center gap-8 text-sm
font-semibold">
          <Link href="/dashboard">
```

```
          <button className="text-gray-300 hover:text-white
transition-colors">Dashboard</button>
        </Link>
        <button className="text-[#40b594] border-b-2
border-[#40b594] pb-1">Post Job</button>
        <Link href="/employer_message">
          <button className="text-gray-300 hover:text-white
transition-colors">Messages</button>
        </Link>
        <Link href="/employer_notification">
          <button className="text-gray-300 hover:text-white
transition-colors">Notification</button>
        </Link>
        <Link href="/subscription">
          <button className="text-gray-300 hover:text-white
transition-colors">Subscription</button>
        </Link>
        <Link href="/employer_setting">
          <button className="text-gray-300 hover:text-white
transition-colors">Settings</button>
        </Link>
      </nav>
      <EmployerNavProfile />
    </header>

    <main className="max-w-7xl mx-auto px-6 md:px-10 py-10">
      <div className="mb-8">
        <p className="text-xs font-bold uppercase
tracking-widest text-[#40b594] mb-1">Hiring</p>
        <h1 className="text-4xl font-extrabold
text-[#071a15]">Post a Job</h1>
        <p className="text-[#4a5a55] font-medium mt-1">
```

```
            Fill in the details below to publish a new role
          </p>
        </div>

        <div className="flex gap-1 bg-white border
border-gray-100 rounded-2xl p-1.5 mb-8 shadow-sm w-fit">
          {(["post", "drafts"] as const).map((tab) => (
            <button
              key={tab}
              onClick={() => setActiveTab(tab)}
              className={`px-6 py-2.5 rounded-xl text-sm
font-extrabold transition-all ${
                activeTab === tab
                  ? "bg-[#051612] text-white shadow-sm"
                  : "text-[#4a5a55] hover:text-[#071a15]"
              }`}
            >
              {tab === "post" ? "Post Job" : `Drafts
(${draftData.length})`}
            </button>
          ))}
        </div>

        {activeTab === "post" && (
          <div className="flex flex-col lg:flex-row gap-6">
            <div className="lg:w-2/3 space-y-6">
              <section className="bg-white p-8 rounded-2xl
shadow-sm border border-gray-100">
                <div className="flex items-center gap-3 mb-7">
                  <div className="w-8 h-8 bg-[#f0f9f6] rounded-xl
flex items-center justify-center border border-[#d1e8e3]">
```

```
                    <Briefcase size={16}
className="text-[#40b594]" />
                  </div>
                  <h2 className="text-lg font-extrabold
text-[#071a15]">Job Details</h2>
                </div>

                <div className="space-y-6">
                  <div>
                    <label className={labelClass}>
                      Job Title <span
className="text-red-500">*</span>
                    </label>
                    <input
                      type="text"
                      value={form.title}
                      onChange={(e) => setForm((p) => ({ ...p,
title: e.target.value }))}
                      placeholder="e.g. Senior Frontend
Developer"
                      className={inputClass}
                    />
                  </div>

                  <div>
                    <label className={labelClass}>
                      Job Category <span
className="text-red-500">*</span>
                    </label>
                    <Combobox
                      placeholder="e.g. Design, Marketing, IT &
Software"
```

```
                    options={categoryOptions}
                    inputClass={inputClass}
                    value={form.category}
                    onChange={(value) => setForm((p) => ({
...p, category: value }))}
                  />
                </div>

                <div className="grid grid-cols-1 md:grid-cols-2
gap-6">
                  <div>
                    <label className={labelClass}>
                      Location <span
className="text-red-500">*</span>
                    </label>
                    <Combobox
                      placeholder="e.g. Phnom Penh, Remote"
                      options={locationOptions}
                      inputClass={inputClass}
                      value={form.location}
                      onChange={(value) => setForm((p) => ({
...p, location: value }))}
                    />
                  </div>

                  <div className="relative">
                    <label className={labelClass}>Work
Arrangement</label>
                    <select
                      value={form.arrangement}
                      onChange={(e) => setForm((p) => ({ ...p,
arrangement: e.target.value }))}
```

```
                        className={selectClass}
                      >
                        <option value="">On-site / Remote /
Hybrid</option>
                        <option>On-site</option>
                        <option>Remote</option>
                        <option>Hybrid</option>
                      </select>
                      <ChevronDown size={16} className="absolute
right-4 top-[42px] text-[#6b7f79] pointer-events-none" />
                    </div>
                  </div>

                  <div className="grid grid-cols-1 md:grid-cols-2
gap-6">
                    <div className="relative">
                      <label className={labelClass}>Employment
Type</label>
                      <select
                        value={form.employmentType}
                        onChange={(e) => setForm((p) => ({ ...p,
employmentType: e.target.value }))}
                        className={selectClass}
                      >
                        <option value="">Select type</option>
                        <option>Full-time</option>
                        <option>Part-time</option>
                        <option>Contract</option>
                        <option>Freelance</option>
                        <option>Internship</option>
                      </select>
```

```
                      <ChevronDown size={16} className="absolute
right-4 top-[42px] text-[#6b7f79] pointer-events-none" />
                    </div>

                    <div className="relative">
                      <label className={labelClass}>Experience
Level</label>
                      <select
                        value={form.experienceLevel}
                        onChange={(e) => setForm((p) => ({ ...p,
experienceLevel: e.target.value }))}
                        className={selectClass}
                      >
                        <option value="">Select level</option>
                        <option>Entry Level</option>
                        <option>Mid Level</option>
                        <option>Senior</option>
                        <option>Lead / Manager</option>
                        <option>Executive</option>
                      </select>
                      <ChevronDown size={16} className="absolute
right-4 top-[42px] text-[#6b7f79] pointer-events-none" />
                    </div>
                  </div>

                  <div className="grid grid-cols-1 md:grid-cols-2
gap-6">
                    <div>
                      <label className={labelClass}>Salary
Minimum</label>
                      <div className="relative">
```

```
                        <span className="absolute left-4 top-1/2
-translate-y-1/2 text-[#6b7f79] font-bold text-sm">$</span>
                        <input
                          type="number"
                          value={form.salaryMin}
                          onChange={(e) => setForm((p) => ({
...p, salaryMin: e.target.value }))}
                          placeholder="0"
                          className={`${inputClass} pl-8`}
                        />
                      </div>
                    </div>
                    <div>
                      <label className={labelClass}>Salary
Maximum</label>
                      <div className="relative">
                        <span className="absolute left-4 top-1/2
-translate-y-1/2 text-[#6b7f79] font-bold text-sm">$</span>
                        <input
                          type="number"
                          value={form.salaryMax}
                          onChange={(e) => setForm((p) => ({
...p, salaryMax: e.target.value }))}
                          placeholder="0"
                          className={`${inputClass} pl-8`}
                        />
                      </div>
                    </div>
                  </div>

                  <div>
```

```
                  <label className={labelClass}>Application
Deadline</label>
                  <input
                    type="date"
                    value={form.applicationDeadline}
                    onChange={(e) =>
                      setForm((p) => ({ ...p,
applicationDeadline: e.target.value }))
                    }
                    className={inputClass}
                  />
                </div>
              </div>
            </section>

            <section className="bg-white p-8 rounded-2xl
shadow-sm border border-gray-100">
              <h2 className="text-lg font-extrabold
text-[#071a15] mb-2">
                Job Description <span
className="text-red-500">*</span>
              </h2>
              <p className="text-xs text-[#6b7f79]
font-semibold mb-5">
                Describe the role, responsibilities, and what
success looks like
              </p>
              <textarea
                rows={10}
                value={form.description}
                onChange={(e) => setForm((p) => ({ ...p,
description: e.target.value }))}
```

```
                placeholder="Write the job description here..."
                className="w-full px-4 py-3 border
border-gray-200 rounded-xl bg-[#f8faf9] text-[#071a15] text-sm
font-medium placeholder-[#9ab0aa] focus:outline-none
focus:ring-2 focus:ring-[#40b594]/30 focus:border-[#40b594]
transition-all resize-none"
              />
            </section>
          </div>

          <div className="lg:w-1/3 space-y-6">
            <section className="bg-white p-7 rounded-2xl
shadow-sm border border-gray-100">
              <h2 className="text-lg font-extrabold
text-[#071a15] mb-2">Requirements</h2>
              <p className="text-xs text-[#6b7f79]
font-semibold mb-5">
                Skills, qualifications, and must-haves
              </p>
              <textarea
                rows={8}
                value={form.requirements}
                onChange={(e) => setForm((p) => ({ ...p,
requirements: e.target.value }))}
                placeholder="e.g. 3+ years React experience"
                className="w-full px-4 py-3 border
border-gray-200 rounded-xl bg-[#f8faf9] text-[#071a15] text-sm
font-medium placeholder-[#9ab0aa] focus:outline-none
focus:ring-2 focus:ring-[#40b594]/30 focus:border-[#40b594]
transition-all resize-none"
              />
            </section>
```

```
            <section className="bg-white p-7 rounded-2xl
shadow-sm border border-gray-100">
              <h2 className="text-lg font-extrabold
text-[#071a15] mb-6">Application Settings</h2>

              <div className="space-y-5">
                <div className="relative">
                  <label className={labelClass}>Application
Platform</label>
                  <select
                    value={form.applicationPlatform}
                    onChange={(e) =>
                      setForm((p) => ({ ...p,
applicationPlatform: e.target.value }))
                    }
                    className={selectClass}
                  >
                    <option value="">Select platform</option>
                    <option value="internal">Apply on
NexHire</option>
                    <option value="external">External
Link</option>
                  </select>
                  <ChevronDown size={16} className="absolute
right-4 top-[42px] text-[#6b7f79] pointer-events-none" />
                </div>

                <div>
                  <label className={labelClass}>External
Link</label>
                  <input
```

```
                    type="text"
                    value={form.externalApplyLink}
                    onChange={(e) =>
                      setForm((p) => ({ ...p,
externalApplyLink: e.target.value }))
                    }
                    placeholder="https://yoursite.com/apply"
                    className={inputClass}
                  />
                </div>

                <div>
                  <label className={labelClass}>Contact
Email</label>
                  <input
                    type="email"
                    value={form.contactEmail}
                    onChange={(e) =>
                      setForm((p) => ({ ...p, contactEmail:
e.target.value }))
                    }
                    placeholder="careers@company.com"
                    className={inputClass}
                  />
                </div>

                <div className="pt-2">
                  <label className="block text-sm
font-extrabold text-[#071a15] mb-3">
                    Visibility
                  </label>
                  <div className="grid grid-cols-2 gap-3">
```

```
                    {["Public Now", "Save as Draft"].map((opt)
=> (
                      <label
                        key={opt}
                        className="flex items-center gap-2.5
bg-[#f8faf9] border border-gray-200 rounded-xl px-4 py-3
cursor-pointer hover:border-[#40b594] transition-all"
                      >
                        <input
                          type="radio"
                          name="status"
                          checked={
                            opt === "Public Now"
                              ? form.status === "active"
                              : form.status === "draft"
                          }
                          onChange={() =>
                            setForm((p) => ({
                              ...p,
                              status: opt === "Public Now" ?
"active" : "draft",
                            }))
                          }
                          className="accent-[#40b594] w-4 h-4"
                        />
                        <span className="text-sm font-bold
text-[#071a15]">{opt}</span>
                      </label>
                    ))}
                  </div>
                </div>
              </div>
```

```
              <div className="border-t border-gray-100 my-6" />

              <div className="space-y-3">
                <button
                  onClick={() => handleSubmit("active")}
                  disabled={loading}
                  className="w-full bg-[#051612] text-white
py-3.5 rounded-xl font-extrabold text-sm hover:bg-[#0d2a23]
transition-all shadow-sm disabled:opacity-60"
                >
                  {loading ? "Publishing..." : "Publish Job"}
                </button>
                <button
                  onClick={() => handleSubmit("draft")}
                  disabled={loading}
                  className="w-full bg-[#f0f4f3] text-[#071a15]
py-3.5 rounded-xl font-extrabold text-sm border border-[#d1e8e3]
hover:bg-[#d1e8e3] transition-all disabled:opacity-60"
                >
                  {loading ? "Saving..." : "Save as Draft"}
                </button>
                <button
                  onClick={() => setForm(emptyForm)}
                  className="w-full text-[#6b7f79] py-2
rounded-xl font-bold text-sm hover:text-[#071a15]
transition-all"
                >
                  Clear Form
                </button>
              </div>
            </section>
```

```
            </div>
          </div>
        )}

        {activeTab === "drafts" && (
          <div className="max-w-3xl">
            <div className="flex items-center justify-between
mb-6">
              <div>
                <p className="text-xs font-bold uppercase
tracking-widest text-[#6b7f79]">
                  Saved drafts
                </p>
                <h2 className="text-xl font-extrabold
text-[#071a15] mt-0.5">
                  {draftData.length} unfinished {draftData.length
=== 1 ? "post" : "posts"}
                </h2>
              </div>
              <button
                onClick={() => setActiveTab("post")}
                className="flex items-center gap-2 bg-[#051612]
text-white px-5 py-2.5 rounded-xl font-bold text-sm
hover:bg-[#0d2a23] transition-all shadow-sm"
              >
                <PlusCircle size={17} />
                New Post
              </button>
            </div>

            {draftLoading ? (
              <div className="space-y-4">
```

```
                {[1, 2].map((i) => (
                  <div key={i} className="bg-white border
border-gray-100 rounded-2xl p-6 animate-pulse">
                    <div className="h-4 bg-gray-100 rounded w-1/3
mb-3" />
                    <div className="h-3 bg-gray-100 rounded
w-1/4" />
                  </div>
                ))}
              </div>
            ) : draftData.length === 0 ? (
              <div className="bg-white rounded-2xl border
border-gray-100 p-12 text-center shadow-sm">
                <div className="w-14 h-14 bg-[#f0f9f6]
rounded-2xl flex items-center justify-center mx-auto mb-4 border
border-[#d1e8e3]">
                  <Briefcase size={24} className="text-[#40b594]"
/>
                </div>
                <p className="font-extrabold text-[#071a15]">No
drafts yet</p>
                <p className="text-sm text-[#6b7f79] mt-1">
                  Save a job as draft and it will appear here.
                </p>
              </div>
            ) : (
              <div className="space-y-4">
                {draftData.map((draft) => (
                  <div
                    key={draft.id}
```

```
                    className="bg-white border border-gray-100
rounded-2xl p-6 flex items-center justify-between group
hover:border-[#40b594] hover:shadow-md transition-all shadow-sm"
                  >
                    <div className="flex items-center gap-5">
                      <div className="w-12 h-12 bg-[#f0f9f6]
rounded-xl flex items-center justify-center border
border-[#d1e8e3] shrink-0">
                        <Briefcase size={22}
className="text-[#40b594]" />
                      </div>
                      <div>
                        <h3 className="text-base font-extrabold
text-[#071a15]">
                          {draft.title}
                        </h3>
                        <div className="flex items-center gap-4
mt-1">
                          <span className="text-xs font-bold
text-[#40b594] bg-[#f0f9f6] px-2.5 py-1 rounded-lg border
border-[#d1e8e3]">
                            {draft.category}
                          </span>
                          <div className="flex items-center
gap-1.5 text-[#6b7f79] text-xs font-semibold">
                            <Clock size={13} />
                            {draft.lastSaved}
                          </div>
                        </div>
                      </div>
                    </div>
```

```
                    <div className="flex items-center gap-2">
                      <button
                        onClick={() =>
handleDeleteDraft(draft.id)}
                        disabled={deletingId === draft.id}
                        className="p-2.5 rounded-xl
text-[#6b7f79] hover:bg-red-50 hover:text-red-500 transition-all
disabled:opacity-40"
                        title="Delete draft"
                      >
                        <Trash2 size={18} />
                      </button>
                      <Link href={`/jobs/${draft.id}`}>
                        <button className="flex items-center
gap-2 bg-[#051612] text-white px-5 py-2.5 rounded-xl font-bold
text-sm hover:bg-[#0d2a23] transition-all">
                          <FileEdit size={16} />
                          View Draft
                        </button>
                      </Link>
                    </div>
                  </div>
                ))}
              </div>
            )}
          </div>
        )}
      </main>
    </div>
  );
};
```

```
export default PostJob;
"use client";

import React from 'react';
import { Check, Layers, FileText, Briefcase, Mail, BarChart2,
Zap, ArrowUpRight } from 'lucide-react';
import Link from 'next/link';

const SubscriptionPlans = () => {
 const plans = [
   {
     name: 'Free',
     badge: 'Normal',
     price: '0',
     description: 'One job post per month. Perfect for casual
hiring.',
     buttonText: 'Current Plan',
     buttonVariant: 'ghost' as const,
     icon: Layers,
     iconBg: 'bg-[#0d2a23]',
     iconColor: 'text-[#40b594]',
     current: true,
     jobSlots: '1 job slot / month',
     features: [
       'Basic job listing',
       'Standard visibility',
       'Applicant dashboard',
     ],
   },
   {
     name: 'Standard',
     badge: 'Popular',
```

```
    price: '4.99',
    description: 'Post up to 3 jobs per month. Best for growing
teams.',
    buttonText: 'Subscribe Standard',
    buttonVariant: 'primary' as const,
    icon: FileText,
    iconBg: 'bg-[#133228]',
    iconColor: 'text-[#40b594]',
    current: false,
    highlight: true,
    jobSlots: '3 job slots / month',
    features: [
      'Everything in Free',
      'Standard visibility boost',
      'Email support',
      'Applicant filtering',
    ],
  },
  {
    name: 'Premium',
    badge: 'Best Value',
    price: '10.99',
    description: 'Post up to 7 jobs per month. For high-volume
hiring.',
    buttonText: 'Go Premium',
    buttonVariant: 'primary' as const,
    icon: Briefcase,
    iconBg: 'bg-[#40b594]',
    iconColor: 'text-[#051612]',
    current: false,
    jobSlots: '7 job slots / month',
    features: [
```

```
        'Everything in Standard',
        'Featured job placement',
        'Priority support',
        'Candidate analytics',
        'Advanced reporting',
      ],
    },
  ];

  return (
    <div className="min-h-screen font-sans pb-16" style={{
background: '#f0f4f3' }}>

      {/* ── HEADER — matches all employer pages ── */}
      <header className="bg-[#051612] text-white px-8 py-4 flex
items-center justify-between sticky top-0 z-50 shadow-lg">
        <div className="flex items-center gap-2.5">
          <img src="/logo.png" alt="NexHire" className="w-8 h-8"
/>
          <span className="text-xl font-extrabold
tracking-tight">NexHire</span>
        </div>

        <nav className="hidden md:flex items-center gap-8 text-sm
font-semibold">
          <Link href="/dashboard">
            <button className="text-gray-300 hover:text-white
transition-colors">Dashboard</button>
          </Link>
          <Link href="/post_job">
            <button className="text-gray-300 hover:text-white
transition-colors">Post Job</button>
```

```
        </Link>
        <Link href="/employer_message">
          <button className="text-gray-300 hover:text-white
transition-colors">Messages</button>
        </Link>
        <Link href="/employer_notification">
          <button className="text-gray-300 hover:text-white
transition-colors">Notification</button>
        </Link>
        <button className="text-[#40b594] border-b-2
border-[#40b594] pb-1">Subscription</button>
        <Link href="/employer_setting">
          <button className="text-gray-300 hover:text-white
transition-colors">Settings</button>
        </Link>
      </nav>

      <Link href="/employer_profile">
        <div className="flex items-center gap-3 cursor-pointer
group">
          <div className="text-right">
            <p className="text-[10px] text-gray-500 uppercase
tracking-widest">Company</p>
            <p className="text-sm font-bold text-white
group-hover:text-[#40b594] transition-colors">Profile</p>
          </div>
          <div className="w-10 h-10 bg-[#40b594] rounded-full
flex items-center justify-center font-extrabold text-[#051612]
text-sm">C</div>
        </div>
      </Link>
    </header>
```

```
     <main className="max-w-7xl mx-auto px-6 md:px-10 py-10">

       {/* ── PAGE TITLE ── */}
       <div className="flex flex-col md:flex-row md:items-end
justify-between gap-6 mb-10">
         <div>
           <p className="text-xs font-bold uppercase
tracking-widest text-[#40b594] mb-1">Plans & Billing</p>
           <h1 className="text-4xl font-extrabold text-[#071a15]
leading-tight">Subscription Plans</h1>
           <p className="text-[#4a5a55] font-medium mt-1">Select
the plan that fits your hiring needs.</p>
         </div>



       </div>

       {/* ── PLAN CARDS ── */}
       <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
         {plans.map((plan, idx) => {
           const Icon = plan.icon;
           const isHighlight = plan.highlight;

           return (
             <div
               key={idx}
               className={`relative rounded-2xl border flex
flex-col overflow-hidden transition-all shadow-sm
hover:shadow-md ${
                 isHighlight
                   ? 'bg-[#051612] border-[#0d2a23]'
```

```
                  : 'bg-white border-gray-100'
              }`}
            >
              {/* Badge */}
              <div className="px-7 pt-7 pb-0 flex items-start
justify-between">
                <span className={`text-xs font-extrabold px-3
py-1 rounded-full ${
                  isHighlight
                    ? 'bg-[#133228] text-[#40b594]'
                    : plan.current
                      ? 'bg-gray-100 text-[#6b7f79]'
                      : 'bg-emerald-50 text-emerald-700 ring-1
ring-emerald-200'
                }`}>
                  {plan.badge}
                </span>
                <ArrowUpRight
                  size={18}
                  className={isHighlight ? 'text-[#40b594]
opacity-60' : 'text-[#6b7f79] opacity-40'}
                />
              </div>

              {/* Icon + Name */}
              <div className="px-7 pt-6 pb-0 flex items-center
gap-4">
                <div className={`${plan.iconBg} p-3
rounded-xl`}>
                  <Icon size={22} className={plan.iconColor} />
                </div>
                <div>
```

```
                    <p className={`text-xs font-bold uppercase
tracking-widest ${isHighlight ? 'text-[#40b594]' :
'text-[#6b7f79]'}`}>
                      {plan.name}
                    </p>
                    <p className={`text-xs font-semibold mt-0.5
${isHighlight ? 'text-gray-400' : 'text-[#9ab0aa]'}`}>
                      {plan.jobSlots}
                    </p>
                  </div>
                </div>

                {/* Price */}
                <div className="px-7 pt-5 pb-0">
                  <div className="flex items-baseline gap-1">
                    <span className={`text-5xl font-extrabold
leading-none ${isHighlight ? 'text-white' : 'text-[#071a15]'}`}>
                      ${plan.price}
                    </span>
                    <span className={`text-sm font-semibold
${isHighlight ? 'text-gray-400' : 'text-[#6b7f79]'}`}>/mo</span>
                  </div>
                  <p className={`text-sm font-medium mt-3
leading-relaxed ${isHighlight ? 'text-gray-300' :
'text-[#4a5a55]'}`}>
                    {plan.description}
                  </p>
                </div>

                {/* Divider */}
                <div className={`mx-7 my-6 h-px ${isHighlight ?
'bg-[#0d2a23]' : 'bg-gray-100'}`} />
```

```
                {/* Features */}
                <div className="px-7 pb-0 flex-1 space-y-3">
                  {plan.features.map((feat, fIdx) => (
                    <div key={fIdx} className="flex items-center
gap-3">
                      <div className={`w-5 h-5 rounded-full flex
items-center justify-center flex-shrink-0 ${
                        isHighlight ? 'bg-[#133228]' :
'bg-[#f0f9f6]'
                      }`}>
                        <Check size={11} className={isHighlight ?
'text-[#40b594]' : 'text-[#40b594]'} />
                      </div>
                      <span className={`text-sm font-medium
${isHighlight ? 'text-gray-300' : 'text-[#4a5a55]'}`}>
                        {feat}
                      </span>
                    </div>
                  ))}
                </div>

                {/* CTA */}
                <div className="p-7">
                  <button
                    className={`w-full py-3.5 rounded-xl
font-extrabold text-sm transition-all shadow-sm ${
                      plan.current
                        ? 'bg-gray-100 text-[#6b7f79]
cursor-default'
                        : isHighlight
```

```
                        ? 'bg-[#40b594] text-[#051612]
hover:bg-[#33997a]'
                        : 'bg-[#051612] text-white
hover:bg-[#0d2a23]'
                  }`}
                  disabled={plan.current}
                >
                  {plan.buttonText}
                </button>
              </div>
            </div>
          );
        })}
      </div>

      {/* ── CURRENT PLAN BANNER ── */}
      <div className="mt-8 bg-white rounded-2xl border
border-gray-100 shadow-sm px-8 py-6 flex flex-col md:flex-row
items-start md:items-center justify-between gap-4">
        <div className="flex items-center gap-4">
          <div className="w-10 h-10 bg-gray-100 rounded-xl flex
items-center justify-center">
            <Zap size={18} className="text-[#6b7f79]" />
          </div>
          <div>
            <p className="text-xs font-bold uppercase
tracking-widest text-[#6b7f79]">Current Plan</p>
            <p className="text-base font-extrabold
text-[#071a15] mt-0.5">Free Tier — 1 active job slot</p>
          </div>
        </div>
        <div className="flex items-center gap-3">
```

```
          <span className="text-xs font-bold
text-[#6b7f79]">Renews automatically · Cancel anytime</span>
          <Link href="/setting">
            <button className="text-xs font-bold text-[#40b594]
hover:underline transition-colors">
              Manage billing →
            </button>
          </Link>
        </div>
      </div>

    </main>
  </div>
 );
};

export default SubscriptionPlans;
```